Project-Based Case Study

# Design and Evaluation of the Technology Template “Template-Projekte”

Technical Investigation of a Reusable Foundation for Web, Cloud, and Desktop Projects


| | |
|---|---|
| **Project:** | Template-Projekte |
| **Author:** | Marcel Tenhaft |
| **Document version:** | 23 August 2026 |


Independently authored technical documentation and evaluation
of the “Template-Projekte” project

## Contents

## List of Figures

## List of Tables

## List of Abbreviations

| Abbreviation | Meaning |
|---|---|
| API | Application Programming Interface |
| AST | Abstract Syntax Tree |
| AWS | Amazon Web Services |
| CI | Continuous Integration |
| CLI | Command Line Interface |
| CORS | Cross-Origin Resource Sharing |
| CSP | Content Security Policy |
| DB | Database |
| DEB | Debian package format |
| E2E | End-to-End |
| GCP | Google Cloud Platform |
| HPC | High Performance Computing |
| HTTP | Hypertext Transfer Protocol |
| JSON | JavaScript Object Notation |
| LOC | Lines of Code |
| PG | PostgreSQL |
| PID | Process Identifier |
| RBAC | Role-Based Access Control |
| SDK | Software Development Kit |
| SQL | Structured Query Language |
| TLS | Transport Layer Security |
| UI | User Interface |
| URL | Uniform Resource Locator |
| ZIP | Compressed archive format |

# 1 Introduction

## 1.1 Background

New software projects repeatedly face the same technical decisions. These concern architecture, technologies, interfaces, configuration, testing, build, and deployment. If these foundations are created anew each time, projects begin from differing technical baselines. At the same time, web, cloud, and desktop applications require both shared structures and platform-specific extensions.

The “Template-Projekte” project consolidates these foundations in a reusable full-stack technology template. It is intended to provide a common basis for web, cloud, and desktop projects (kleiveist, 2026s). Such a basis can consolidate recurring decisions and establish a consistent development model. To achieve this, shared components must be clearly separated from platform-specific extensions.

## 1.2 Problem Statement

Repeatedly initializing similar projects consumes effort outside product development. Without a shared baseline, structure, tools, configuration, and build processes differ; separate starting points increase maintenance effort and may diverge technically.

The first baseline examined already had separate runtime components, tests, builds, CI, and documented architectural boundaries. However, rules for size, complexity, and selected dependencies were not operationalized through a shared repository-wide gate. For example, the parent state `0cbe1ba` of the later governance commit contained direct imports of SQLAlchemy and `app.db` in a FastAPI router; `461fc751` moved the readiness logic into a service and infrastructure wiring into the composition root (kleiveist, 2026i). The gap is therefore observable not only in a target description but also in a corrected layer violation.

In short change cycles, including AI-assisted development, a rule that exists only in documentation can become visible late. This observation does not support a general claim about the quality of AI-generated code; it establishes a need for early, reproducible feedback in the development workflow examined.

The template must therefore combine standardization with flexibility: a common basis should enable reuse without burdening projects with unnecessary components. This case study examines how project profiles and optional extensions address this trade-off and where adaptation is still required (kleiveist, 2026j, 2026k), and how the governance added later respects this variability.

## 1.3 Objectives

The case study examines the architecture, shared and profile-specific components, and the development and quality-assurance workflows of the “Template-Projekte” project (kleiveist, 2026j, 2026t). Following the original technical acceptance, it includes *Code Quality & Architecture Governance v1*, introduced on 17 August 2026, as a subsequent extension. The study examines central configuration, scanners, the rule and severity model, time-limited exceptions, reporters, language adapters, architecture checks, the CLI, and CI (kleiveist, 2026c, 2026i).

Based on existing project artifacts and evidence, it assesses reusability, standardization, productivity potential, and suitability for different project types. From this assessment, it derives criteria for use and selection, technical limitations, and external prerequisites (kleiveist, 2026r). The study does not investigate universal suitability for every class of software or a product-specific domain architecture. Nor is the gate intended to prove clean code, lower maintenance costs, or a reduced defect rate. The narrower objective is to establish which defined violations the examined state detects reproducibly and how it handles them.

## 1.4 Research Questions

Six questions structure the investigation and assessment:

1. Which functional and technical, quality, and operational requirements does the technology template address?
2. How do the architecture and profile model support the provision of different variants for web, cloud, and desktop projects?
3. To what extent do the shared technical core and the single-repository strategy promote reuse and standardization across projects?
4. What potential does the template offer for reducing recurring effort in project initialization, development workflows, automated code-quality and architecture checks, and deployment?
5. For which project types and customer requirements does the template provide a suitable technical starting point?
6. Which technical limitations, maintenance efforts, risks, and external dependencies must be considered when using it, including its governance?

Their answers are based on requirements, architecture, and the available verification evidence. As a cross-cutting evaluation aspect, the study asks how the profile-based full-stack template enforces selected code-quality and architecture rules across languages, configurably and through CI, without moving them into the product runtime. This aspect complements the six questions and is not treated as an independent, empirically generalizable main research question.

## 1.5 Methodology

The investigation follows a technical case-study approach (Runeson & Höst, 2009). An inventory records the structure, components, and boundaries of responsibility within the master repository. The subsequent analysis binds the observed state to branch `main`, version `0.1.0`, and commit `461fc7519e0db638330904a7496c488e8a0d18bc`; the working tree was clean at the outset. It compares the governance commit with its parent `0cbe1ba` in a before-and-after analysis (kleiveist, 2026i, 2026s).

The architecture analysis examines the frontend, backend, desktop integration, interfaces, persistence, and deployment. It distinguishes substantiated properties from assumptions and unresolved verification points (kleiveist, 2026j).

The profile analysis examines the shared core, variants, and optional capabilities. In addition, the Python tooling is examined with respect to generation, system checks, installation, development, testing, builds, and release (kleiveist, 2026k, 2026t). Internal project descriptions of the architecture, profiles, and tools serve as primary sources for the documented state.

For the governance, configuration, implementation, boundary and regression tests, CLI exit codes, workflow files, local command output, and current CI runs are cross-checked. An internal evidence matrix maps each central claim to repository, test, and, where applicable, CI evidence and a target chapter. The claim that 901 code LOC normally produces an error therefore rests on the configuration, classification, the 900/901 transition test, and the non-ignored result of the quality command; identified counterexamples are reported as limitations.

On 22 August 2026, help commands, focused and complete quality runs, 333 directly executable tooling and case-study tests, and the public paths `doctor`, `test --suite all`, `build web`, and `docs index --dry-run` were executed. The technical baseline remained commit `461fc751`; the case-study tests simultaneously checked the emerging, not-yet-versioned German document state. Missing system libraries, services, or toolchains are classified neither as success nor as a test of the disabled subject. External or product-specific checks remain separate. This evidence complements the earlier acceptance (kleiveist, 2026r). Figure 1 shows the sequence of the investigation. Assessment and transfer therefore take place only after the inventory, analysis, and verification; unresolved verification points are not treated as confirmed properties.

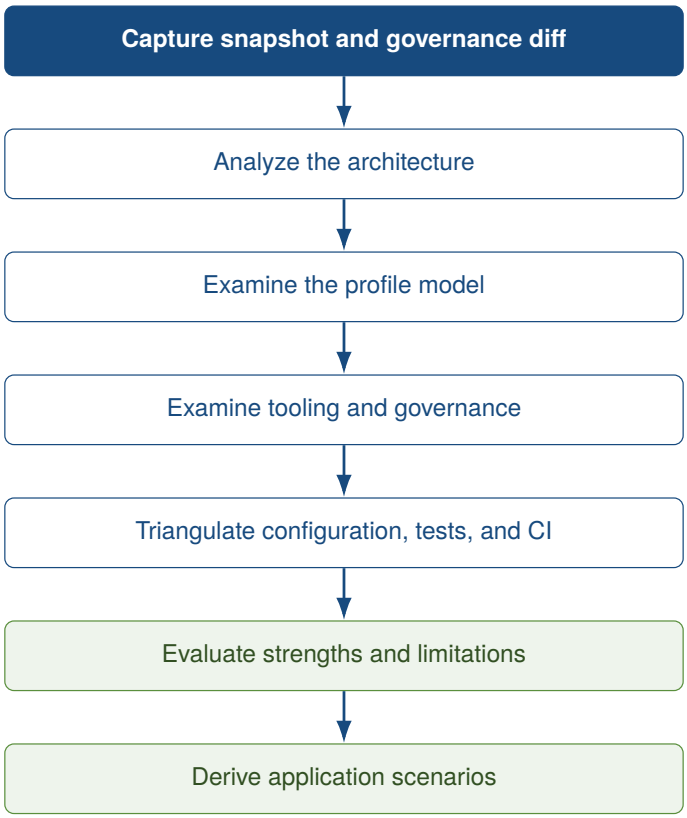


Figure 1: Methodological steps of the case study

Source: Author's own illustration.

## 1.6 Structure of the Case Study

The requirements are followed by the architecture, project profiles, and tooling. The case study then addresses quality assurance and verification, assesses the observed state, and transfers the findings to customer projects. The conclusion answers the research questions and identifies limitations and prospects for further development.

# 2 Requirements for the Technology Template

## 2.1 Target Vision for the Technology Template

The template is intended to provide a reusable basis for web, cloud, and desktop projects. To this end, it combines planned commonality with controlled variability (Clements & Northrop, 2001; Krueger, 1992). A shared core is intended to centralize structure, workflows, tooling, documentation, configuration, tests, and interfaces.

Profiles select suitable combinations from this core; optional capabilities such as PostgreSQL extend compatible backend profiles (kleiveist, 2026k). The frontend, backend, desktop layer, persistence, tooling, and deployment remain separate areas of responsibility. At this stage, these statements describe requirements, not verification results.

## 2.2 Requirements for Reusability and Standardization

The baseline is intended to define shared structures and public workflows while allowing controlled profile variants. Centralized maintenance prevents copies from diverging. Table 1 summarizes the verifiable requirements.

Table 1: Key requirements for the technology template

| ID | Requirement | Operationalized objective | Basis for subsequent verification |
|---|---|---|---|
| A-01 | Baseline | It must be possible to generate the basic structure. | Project generation |
| A-02 | Structure | Responsibilities are assigned to consistent locations. | Repository structure |
| A-03 | Profiles | Only designated components are activated. | Profile model |
| A-04 | Workflows | Checks, initialization, installation, startup, testing, and building share common entry points. | Tooling |
| A-05 | Maintainability | Shared components are maintained centrally. | Repository strategy |
| A-06 | Traceability | Changes and configuration are versioned and documented. | Version state, documentation |
| A-07 | Extensibility | Optional capabilities have documented dependencies. | Capability model |
| A-08 | Quality policy | Measurable rules are centrally configured and can be evaluated locally and in CI. | Configuration, CLI, workflow |
| A-09 | Architecture governance | Selected layer and feature boundaries are checked automatically. | Import analysis, architecture tests |
| A-10 | Proportionality | Findings remain visible; only errors and required failed tools block. | Severity, exception, and exit-code tests |

Source: Author's own compilation.

The stated verification basis is subsequently used to assess these target criteria. A-08 through A-10

originated only with the governance extension and are therefore not attributed retrospectively to the original baseline.

## 2.3 Requirements for Web, Cloud, and Desktop Applications

All variants are intended to share the frontend foundation, structure, configuration logic, tests, and documentation. Browser projects do not require a desktop layer and need a backend only when necessary. Cloud variants require an API, deployment boundaries, and health and readiness checks. Desktop variants use the same frontend in a native runtime; combined variants add a backend and, where applicable, browser access.

Table 2: Requirements matrix for the supported project types

| Requirement | Web | Cloud | Local desktop | Desktop + Cloud |
|---|---|---|---|---|
| Frontend | required | required | required | required |
| Backend / API | optional | required | not inherently required | required |
| Desktop runtime | no | no | required | required |
| Deployment boundary | optional | required | no | required |
| Persistence | optional | optional | product-specific | optional |

Source: Author's own compilation based on kleiveist (2026k).

PostgreSQL is an optional backend capability, not a platform profile; profiles determine neither the format, persistence provider, nor source of truth (kleiveist, 2026k, 2026l). Client and server configuration remain separate; no cloud platform is assumed.

## 2.4 Quality and Maintainability Requirements

The requirements are guided by verifiable characteristics of ISO/IEC 25010 (ISO/IEC, 2023). Components and profiles should have clear responsibilities, be testable in isolation, and remain maintainable through documented decisions. Extensions and technology updates should not require a copy of the entire baseline, although they still entail risks.

Identical inputs and configurations should produce traceable generation and build results. This strengthens the verifiable relationship between the source-code state and the artifact (Lamb & Zacchiroli, 2022). It should be possible to generate target artifacts in a controlled manner, and configuration priorities should be comprehensible; server-side secrets remain separate from client configuration.

Governance v1 functionally adds the loading and validation of a versioned policy, selection of supported source files, code and physical-line counts, scope metrics, architecture rules, exclusions, temporary exceptions, and text and JSON reporting. Findings should identify the path, optional line and symbol, rule ID and name, severity, actual value, threshold, explanation, and remediation. Non-functional requirements include a shared local and CI entry point, quality logic separated from the dispatcher, controlled failures for invalid configuration, and compatibility with generated profiles (kleiveist, 2026c).

Table 3 shows the implemented default classification. *WARNING* and *STRONG_WARNING* provide

information without changing the exit code by themselves; *ERROR* blocks unless an applicable exception exists.

Table 3: Project-specific thresholds in Governance v1

| Metric | PASS | WARNING | STRONG_WARNING | ERROR |
|---|---|---|---|---|
| Code LOC per file | 0–600 | 601–750 | 751–900 | 901 or more |
| Physical lines | 0–1200 | 1201 or more | – | – |
| Code LOC per function | 0–50 | 51–80 | 81–120 | 121 or more |
| Code LOC per class | 0–300 | 301–500 | 501–700 | 701 or more |
| Cyclomatic complexity | 1–10 | 11–15 | 16–20 | 21 or more |
| Nesting depth | 0–3 | 4 | 5 | 6 or more |
| Parameters per function | 0–5 | 6–8 | 9–10 | 11 or more |
| FastAPI handler, code LOC | 0–50 | – | – | 51 or more |

Source: Author's own compilation based on kleiveist (2026a).

The 900-code-line boundary is a project-specific governance decision, not a universal scientific clean-code threshold: 900 LOC are permitted but already fall in the strong-warning band; 901 LOC are normally classified as `CQ001 ERROR`. Likewise, 120 function lines, 700 class lines, complexity 20, five nesting levels, and ten parameters operationalize this repository's policy. McCabe's source establishes the cyclomatic metric, not the threshold of 20 chosen here (McCabe, 1976).

The file limit is an outer safety net. A file with 850 LOC may be problematic despite satisfying the hard limit; a file with 200 LOC may still exhibit high complexity, unfavorable dependencies, or mixed responsibilities. The metrics are automatable indicators and replace neither domain-aware code review nor an empirical maintainability measurement. The technical suppressibility of `CQ001` is therefore assessed separately in Section 7.3.

### 2.5 Scope

The template is not a universal software platform. Its target scope encompasses recurring web, backend, and desktop projects, including full-stack business applications.

Business rules, domain and customer data models, and product-specific authentication, role, and authorization concepts are not part of the baseline. It also prescribes no product or user-data format, persistence provider, or cloud provider (kleiveist, 2026j, 2026l).

Highly platform-specific native applications, games, and highly specialized real-time systems fall outside the immediate target scope.

## 3 Architecture of the Technology Template

### 3.1 Overall Architecture

The baseline consists of a master repository with a shared core and declarative variants. Profiles select reusable features; optional capabilities extend only compatible profiles. This controlled variability avoids separate copies of the template (Clements & Northrop, 2001; kleiveist, 2026k).

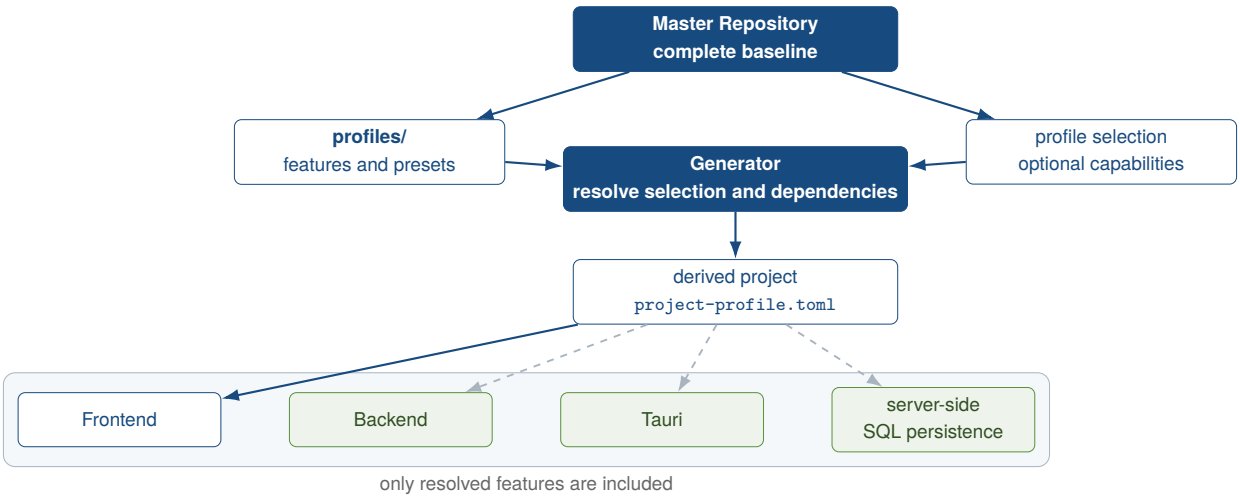


Figure 2: Overall architecture of the technology template

Source: Author's own illustration based on kleiveist (2026j, 2026k, 2026l).

Figure 2 distinguishes the baseline from the derived project. The generator resolves the profile and capabilities, copies the associated paths, and records the result in `project-profile.toml`. The frontend, backend, Tauri, and server-side SQL persistence remain independent components, some of which are optional (kleiveist, 2026j, 2026l).

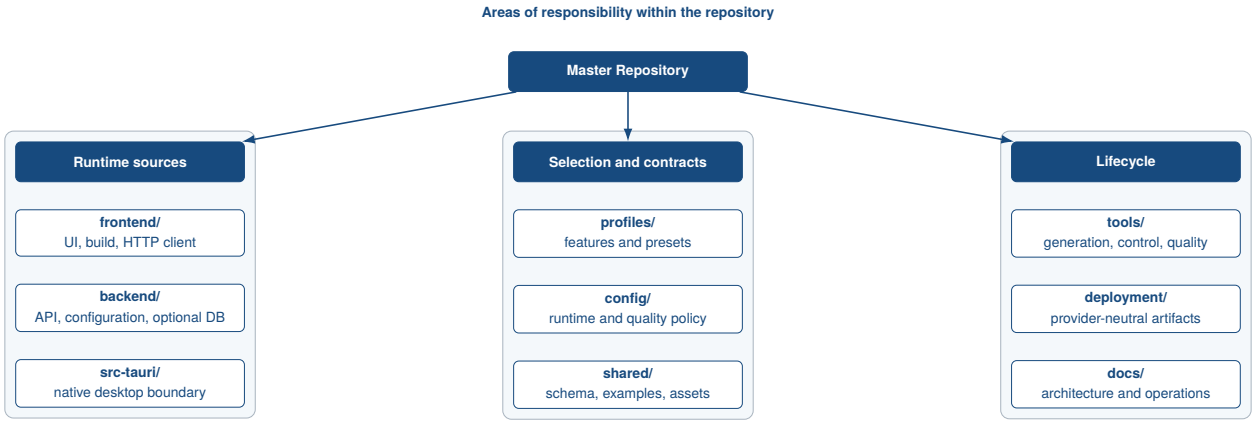


Figure 3: Components and responsibility boundaries in the Master Repository

Source: Author's own illustration based on kleiveist (2026j) and kleiveist (2026m).

The separation of components in Figure 3 keeps the client, server, native integration, and shared artifacts distinct. Profiles and configuration describe the structure and runtime values; tooling and deployment control the components outside the product runtime. Since `461fc751`, quality governance has belonged to this development and CI area. It examines runtime sources but is neither shipped nor made part of the HTTP, Tauri, or persistence data flow (kleiveist, 2026s).

Local developers, coding agents, and GitHub Actions invoke `tools/control.py`. The dispatcher delegates to the separate quality orchestration, which combines central policy, repository scanning, language-specific tools, backend and frontend architecture checks, exception handling, and reporting. Figure 15 presents this control flow in Section 6.1. The architecture decision adds a verification layer to the existing runtime architecture, not another product component.

The governance commit also corrected the health router's previous direct coupling to SQLAlchemy and `app.db`: the router now delegates to a service, while concrete infrastructure is wired in `backend/app/main.py`. This is an observable before-and-after case for the subsequently automated backend boundaries (kleiveist, 2026i).

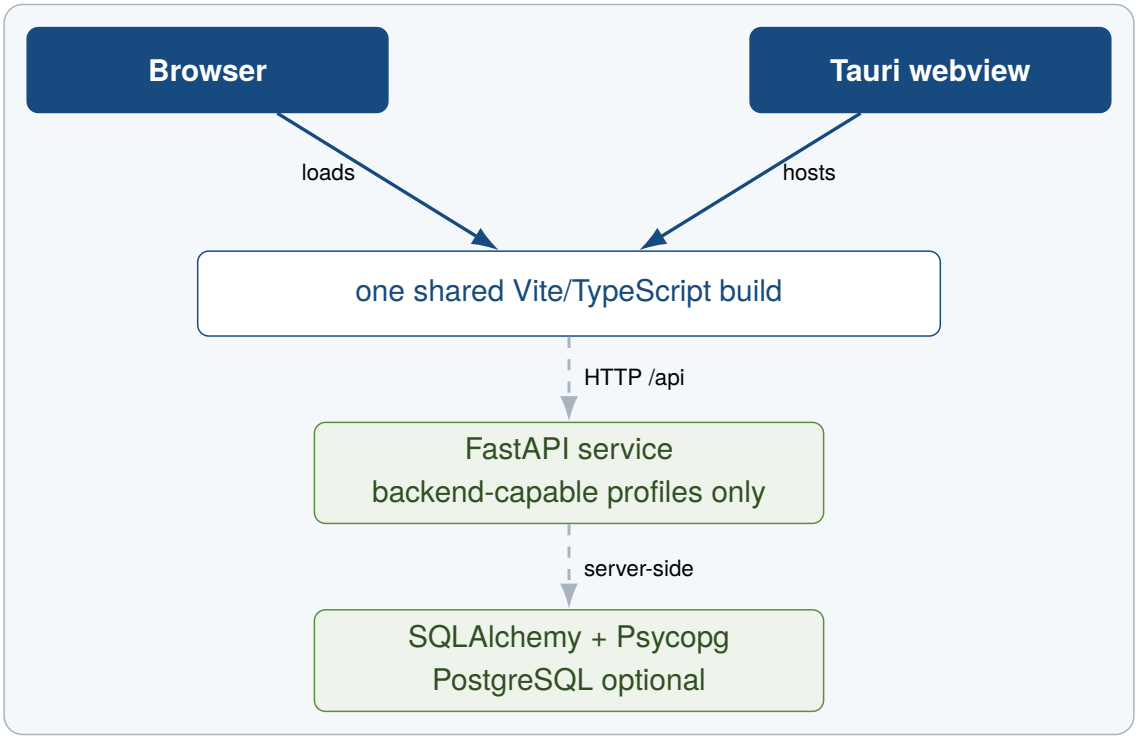


Figure 4: Profile-dependent runtime architecture

Source: Author's own illustration based on kleiveist (2026j) and kleiveist (2026k).

The browser or a Tauri webview loads the same Vite build (Figure 4). Backend-enabled profiles use FastAPI over HTTP; only the backend accesses the optional PostgreSQL persistence shown. Local product storage is separate and is not implemented (kleiveist, 2026l).

## 3.2 Frontend with Vite and TypeScript

The frontend is the shared presentation layer for all five profiles. Vite is used for development and building; TypeScript checks the strictly configured source code, while Vitest tests the HTTP client (Microsoft, 2026; VoidZero Inc. and Vite Contributors, 2026).

The generated profile module enables backend access only in appropriate variants. The browser and Tauri use the same source files and a public `VITE_API_BASE_URL`. Server logic, secrets, and direct database access remain outside the client; the web-only and desktop-local profiles require no backend (kleiveist, 2026j, 2026k).

Table 4 documents the areas and versions of the historical baseline and the added v1.0.0 release tools.

Table 4: Technology stack of the baseline and the v1.0.0 candidate state

| Technology | Role in the template | Version / source | Project evidence |
|---|---|---|---|
| Node.js | frontend tools and CI runtime | major version 24 | `README.md`, `.github/workflows` |
| Vite | development server and frontend build | range from 6.0.7; lock: 6.4.2 | `frontend/package.json`, `frontend/package-lock.json` |
| TypeScript | static checking of the frontend | range from 5.7.3; lock: 5.9.3 | `frontend/package.json`, `frontend/tsconfig.json` |
| FastAPI | HTTP API and validation | $\geq$0.111, $<$1; production: 0.111.0 | `backend/requirements.txt`, `backend/requirements-production.txt` |
| Tauri | desktop webview and packaging boundary | major version 2; lock: 2.11.5 | `src-tauri/Cargo.toml`, `src-tauri/Cargo.lock` |
| Rust | native composition layer and analyzer build | Tauri: at least 1.77, edition 2021; analyzer: rustc 1.97.1, edition 2024 | `src-tauri/Cargo.toml`, `tools/quality/rust_analyzer/rust-toolchain.toml` |
| Syn/WASI/Wasmtime | portable Rust scope and control-flow analysis | Syn 2.0.119; proc-macro2 1.0.107; `wasm32-wasip1`; Wasmtime 47.0.1 | `tools/quality/rust_analyzer`, `tools/requirements.txt` |
| PostgreSQL | optional relational runtime unit | container: 16.15-alpine3.24 | `deployment/compose.yaml` |
| SQLAlchemy | engine, session, and model foundation | $\geq$2, $<$3; production: 2.0.36 | `backend/requirements-database*.txt` |
| Alembic | explicit schema migrations | $\geq$1.13, $<$2; production: 1.14.0 | `backend/requirements-database*.txt` |
| Psycopg | PostgreSQL driver | $\geq$3.2, $<$4; production: 3.2.3 | `backend/requirements-postgres*.txt` |

Source: Author's own compilation based on the historical baseline (kleiveist, 2026s), the current release manifests, and the verified analyzer provenance.

### 3.3 Backend with FastAPI

The backend forms a separate HTTP layer for the web-cloud, desktop-cloud, and full-platform profiles. Under `/api`, FastAPI currently provides only `/health` and `/ready`. Product-specific services, domain models, and authentication are not prescribed (kleiveist, 2026j; Ramírez, 2026).

Liveness reports the process state. When the database capability is active, readiness additionally checks the connection and responds with HTTP 503 on failure without disclosing internal details. The router, settings, and dependency are separately testable; service and domain modules are added only by the product.

The units remain separate in deployment (Figure 5): the Vite build can be deployed as static content, while FastAPI runs in its own unprivileged container. Compose represents this provider-neutral boundary locally; PostgreSQL is added only when explicitly selected (kleiveist, 2026h).

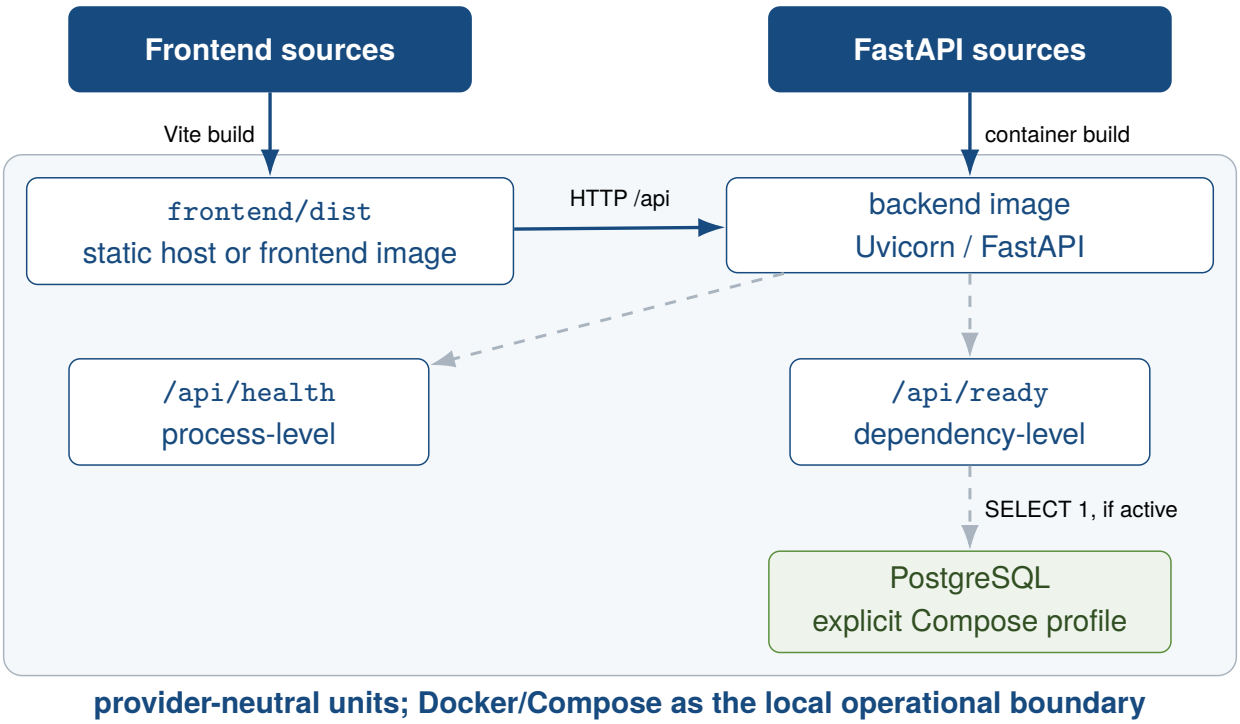


Figure 5: Provider-neutral deployment architecture

Source: Author's own illustration based on kleiveist (2026h).

### 3.4 Desktop Integration with Tauri and Rust

Tauri provides a native boundary without duplicating the frontend. During development, it uses the Vite server; in the build, it uses `frontend/dist`. The Rust entry point only creates the Tauri application; native domain commands and a FastAPI sidecar are absent from the baseline (Klabnik et al., 2026; kleiveist, 2026j; Tauri Contributors, 2026b).

The default capability grants only `core:default`; a Content Security Policy restricts resources and development endpoints. Additional native permissions must be added on a product-specific basis (Tauri Contributors, 2026a). In the desktop-local profile, local functionality resides in the frontend or in future Tauri commands; desktop-cloud and full-platform use the public FastAPI URL. Packaging, signing, and the choice between a remote API, sidecar, or local commands remain product decisions.

### 3.5 Shared Contracts and Interfaces

`shared/` is included in every generated project. It contains static assets, a JSON Schema conforming to Draft 2020-12, and valid and invalid test examples. The contents are serializable and framework-neutral, but define neither a product API nor executable framework code (kleiveist, 2026j, 2026s).

However, the runtime does not use the schema automatically: neither the frontend nor the backend imports it. The health contract is likewise mirrored manually as a TypeScript interface and a FastAPI response. Without code generation or synchronization, contract changes must be propagated on both sides and in consumer tests. The documented target role of shared contracts is therefore not yet fully implemented.

## 3.6 Configuration Model

Project structure and runtime configuration are separate. The project manifest determines the features and is stored in `project-profile.toml`. The variable contract is defined in `config/environment.toml`. For active features, the priority order is CLI override, process environment, local `.env`, and contract default. Derived values also remain traceable (kleiveist, 2026q).

The third central configuration type, `config/code-quality.toml`, is not part of runtime resolution. It defines the schema version, supported extensions, exclusions, thresholds, layer mappings, and exceptions for the development and CI gate (kleiveist, 2026a). The product profile, runtime values, and quality policy thus remain separate contracts. The quality engine does not, however, read the profile manifest itself; its component selection follows the generated directory structure that is present.

Figure 6 shows the security boundaries. Vite receives only the approved `VITE_API_BASE_URL`; FastAPI processes server values with explicit types and masks `DATABASE_URL`. The tooling forwards only relevant values and removes known secret names from the Tauri environment. Invalid or missing values result in controlled termination without exposing secrets.

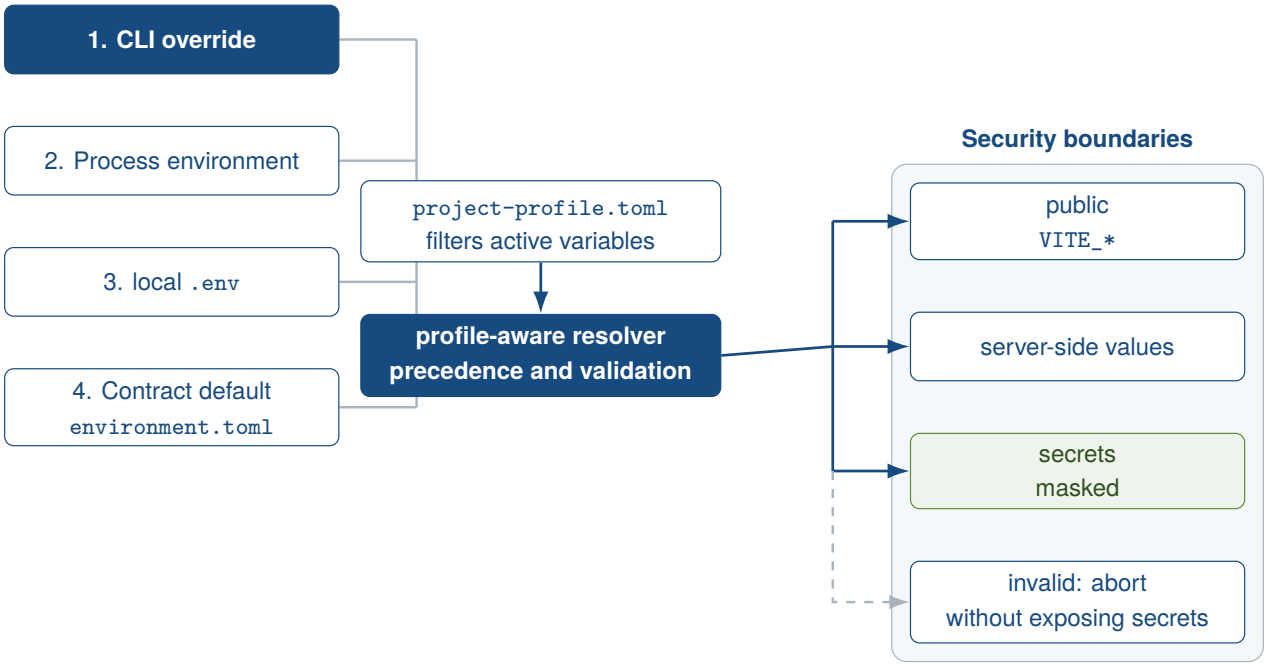


Figure 6: Configuration precedence and security boundaries

Source: Author's own illustration based on kleiveist (2026q).

## 3.7 Server-Side PostgreSQL Persistence with SQLAlchemy and Alembic

PostgreSQL is optional and is not a separate profile. `postgres` requires `database` and thus a backend; the web-only and desktop-local profiles are excluded. The database capability comprises the SQLAlchemy engine, sessions, and Alembic. PostgreSQL adds Psycopg 3, URL requirements, and integration tests and remains a separate runtime unit (kleiveist, 2026g; PostgreSQL Global Development Group, 2026).

The engine is created only upon access and validates connections before use; sessions are closed in a controlled manner. The empty declarative `Base` is an extension point for domain models (SQLAlchemy

Authors and Contributors, 2026).

Alembic uses the same metadata basis for online and offline migrations. Without domain models, there is no initial revision. Neither `create_all()` nor automatic migrations at FastAPI startup are implemented; changes require explicit Alembic commands (Bayer, 2026; kleiveist, 2026g). `/api/health` remains independent of the database, while `/api/ready` checks reachability when the database capability is enabled.

### 3.8 Provider-Neutral Persistence Strategy for Product and User Data

The baseline separates template and development data, runtime configuration, and product and user data. `profiles/`, `project-profile.toml`, `tools/`, and `shared/` describe, generate, or verify the project. `.env`, `config/environment.toml`, `DATABASE_URL`, and API configuration instead determine how an instance runs. Documents, planning states, domain records, media, project files, and histories arise only through product use and require their own area of responsibility and lifecycle. Configuration files are not user-data storage; version-controlled template assets are not runtime assets of the finished product.

A persistence decision distinguishes provider-neutral local file storage, embedded databases, remote databases, and asset storage. Not every product requires all four classes. Local files may use JSON, Markdown, or product-specific formats; an embedded database could, for example, use SQLite. In contrast, the existing `database` and `postgres` capabilities implement server-side SQL persistence with SQLAlchemy, Alembic, and PostgreSQL. Asset storage for images, documents, graphics, or binary files remains separate from structured data. `local-files`, `embedded-database`, `sqlite`, `object-storage`, and synchronization are possible extension paths, not implemented capabilities (kleiveist, 2026l).

Each product project must identify the authoritative data source. In a local application, a local store may be the source of truth; in a server-centered application, it may be a central database. Offline or local-first systems additionally require explicit rules for synchronization direction, revisions, and conflicts; the baseline provides no synchronization engine. The product decision also covers the schema and versioning strategy, migration, backup and recovery, and security boundaries.

The template therefore standardizes neither the data itself, a universal format, nor a general storage provider. It standardizes the boundaries of responsibility and the decisions that must be documented for handling data. Storage format and persistence architecture, platform profile and storage provider, configuration and user data, and asset storage and structured data storage remain separate decisions.

## 4 Project Profiles and Feature Model

### 4.1 Basic Principle of the Profile Model

The master repository manages variants through features, thereby supporting systematic reuse without constituting a complete software product line (Clements & Northrop, 2001; Krueger, 1992). Here, *Core* denotes the paths included in every product project. A *Feature* groups the paths and depen-

dencies of a technical capability, while a *profile* represents a permissible combination of features. A *capability* optionally extends a profile and is not itself a platform profile. The *manifest* records the selection in the generated project (kleiveist, 2026k, 2026s). The Core includes version information, configuration, documentation, profile descriptions, shared artifacts, and tooling, among other elements. Technical runtime components, by contrast, reside in the selected feature paths.

Governance v1 extends the Core with `AGENTS.md`, `config/code-quality.toml`, the quality documentation, and `tools/quality/`; the generator copies these paths into all five profiles (kleiveist, 2026c). Component-dependent checks are derived from the generated structure: frontend tools require an existing `frontend/package.json`, Rust checks require `src-tauri/Cargo.toml`, and architecture checks require the corresponding source path. A normally generated `web-only` project therefore needs no Rust toolchain.

This compatibility is structure-based, not manifest-based. A subsystem that is active in the manifest but accidentally absent can be reported as "not enabled" with a successful check; the reporter has no dedicated `SKIP` status. Manifest consistency remains the responsibility of the doctor, profile, and generator checks. The claim that the gate respects profiles therefore applies to normally generated scaffolds, not arbitrary inconsistent directory states.

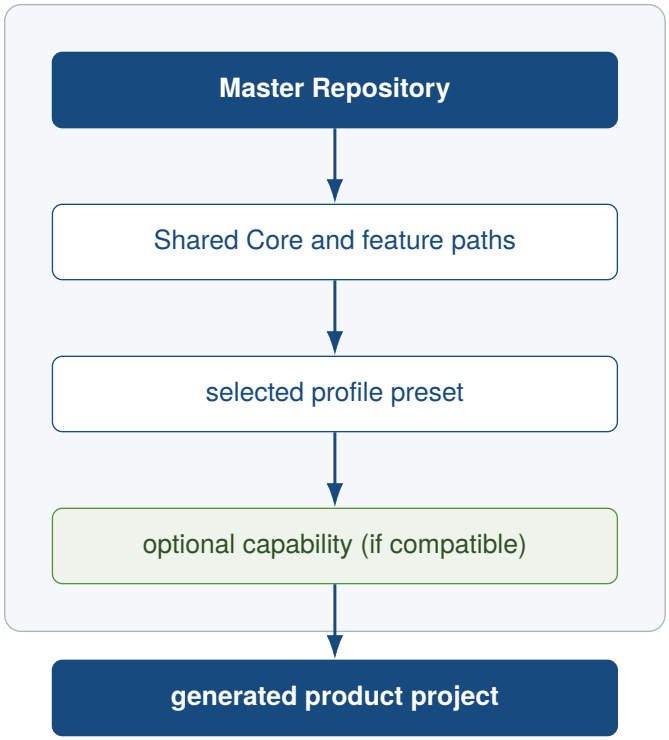


Figure 7: Deriving a product project from a shared template baseline

The generator resolves the profile and compatible capabilities and copies the Core and active feature paths into a new product project (Figure 7). Inactive directories are not selected; web profiles additionally omit the Tauri CLI. The name, slug, and, for Tauri, the application ID can be customized (kleiveist, 2026k, 2026t). Resolution produces an ordered scaffold plan. As a result, the target project does not merely contain disabled components; it predominantly contains only the directories intended for its variant.

Backend and cloud occur together in the current presets. For desktop projects, an additional browser

channel distinguishes `full-platform` from `desktop-cloud`. Capabilities are checked only after the profile has been selected; the resolver does not silently add missing platform features (Figure 8).

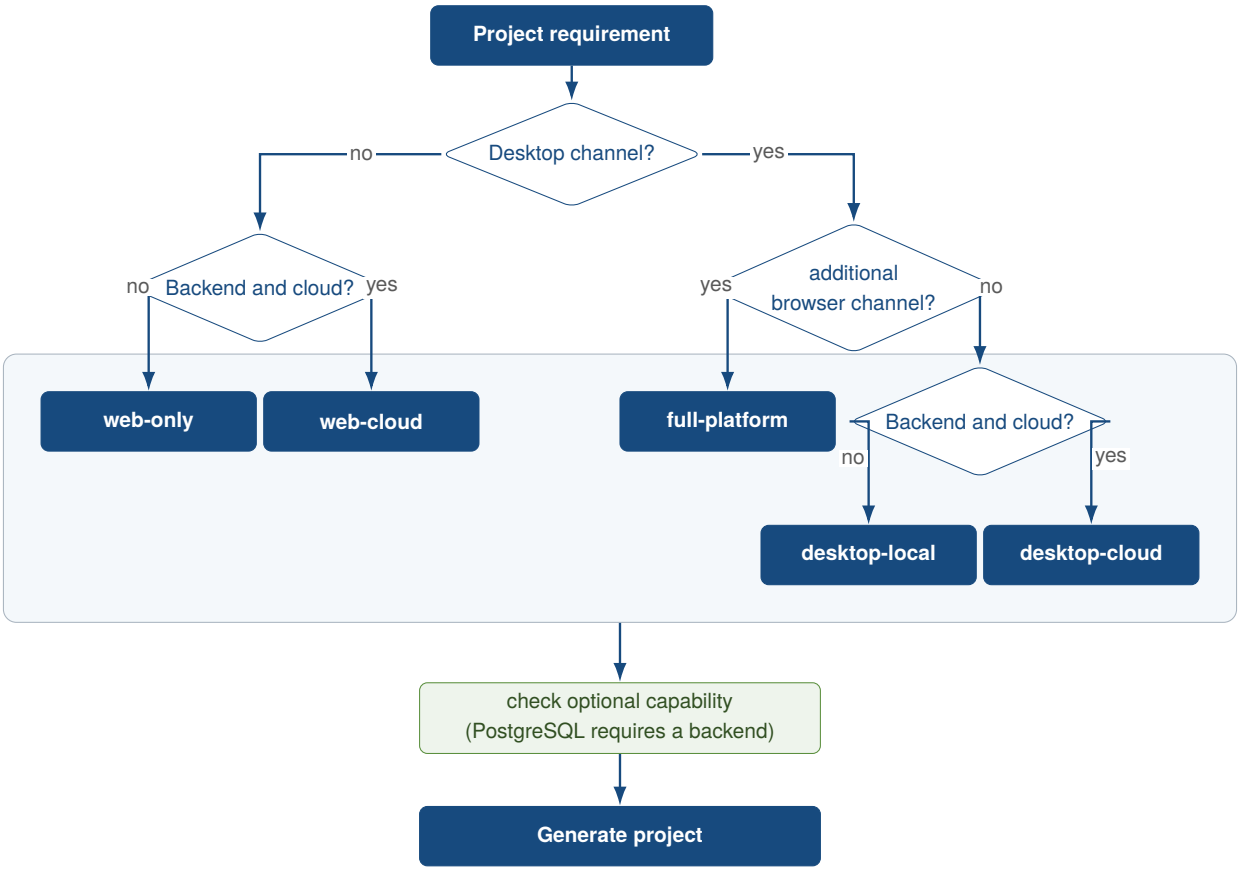


Figure 8: Decision flow for profile and subsequent capability selection

The generated `project-profile.toml` records the schema version, profile identity, requested extensions, and fully resolved features. The tooling validates it and handles only active services and workflows. The master manifest uses `full-platform + postgres` without modifying the preset itself. Runtime values, ports, URLs, and secrets instead belong to the separate configuration model (kleiveist, 2026q, 2026s). Requested optional features and the resolved result remain separately visible in the manifest. This distinction makes transitive dependencies traceable.

Figure 9 and Table 5 compare the five presets. None enables PostgreSQL by default.

| Profile | Frontend | FastAPI | Tauri | Cloud | PostgreSQL |
|---|---|---|---|---|---|
| web-only | included | – | – | – | incompatible |
| web-cloud | included | included | – | included | optional |
| desktop-local | included | – | included | – | incompatible |
| desktop-cloud | included | included | included | included | optional |
| full-platform | included | included | included | included | optional |

Figure 9: Structural feature allocation of the five profile presets

Source: Author's own illustration based on the feature catalog and the profile definitions in the examined project state (kleiveist, 2026k, 2026s).

Table 5: Declared baseline features and resulting runtime directories

| Profile | Baseline features | Runtime directories | PostgreSQL |
|---|---|---|---|
| `web-only` | `frontend` | `frontend/` | incompatible |
| `web-cloud` | `frontend`, `backend`, `cloud` | `frontend/`, `backend/`, `deployment/` | optional |
| `desktop-local` | `frontend`, `tauri` | `frontend/`, `src-tauri/` | incompatible |
| `desktop-cloud` | `frontend`, `backend`, `tauri`, `cloud` | `frontend/`, `backend/`, `src-tauri/`, `deployment/` | optional |
| `full-platform` | `frontend`, `backend`, `tauri`, `cloud` | `frontend/`, `backend/`, `src-tauri/`, `deployment/` | optional |

## 4.2 Web-only Profile

`web-only` adds only the Vite frontend to the Core. The browser loads it without a FastAPI, desktop, deployment, or database layer; the Tauri script and CLI are removed. The profile is therefore suitable for pure client applications without backend access (kleiveist, 2026k). Compared with `web-cloud`, it consequently lacks the API and cloud feature; the generated frontend also does not enable a backend status request.

## 4.3 Web-cloud Profile

`web-cloud` combines a Vite frontend, a separate FastAPI backend, and provider-agnostic deployment files. It has no native desktop layer (kleiveist, 2026h, 2026k). The database module, Alembic, and Psycopg are added only with the compatible PostgreSQL capability, which must be selected separately. The frontend accesses the separate backend through a public URL. Without the capability, the profile remains database-free despite its cloud designation.

## 4.4 Desktop-local Profile

`desktop-local` runs the shared frontend in a Tauri webview. Backend, deployment, and database paths are absent (kleiveist, 2026k). Thus, *local* denotes the absence of a cloud API; no persistence provider is automatically part of the profile (kleiveist, 2026l). Because `database` requires a backend, PostgreSQL is incompatible. Local domain functionality must therefore reside in the frontend or in Tauri commands added for the specific product.

## 4.5 Desktop-cloud Profile

`desktop-cloud` connects the Tauri frontend to the separate FastAPI backend through a public URL and adds deployment files. PostgreSQL is compatible but remains optional; neither a local store nor PostgreSQL is established as the source of truth (kleiveist, 2026h, 2026k, 2026l).

The technical base features correspond to `full-platform`; the profile ID, however, identifies the desktop client as the intended product channel. The distinction is semantic: it does not introduce an additional technical component.

## 4.6 Full-platform Profile

`full-platform` provides the browser and Tauri as two product channels for the same frontend base; both access the same FastAPI backend. Its scaffold differs from `desktop-cloud` only in profile identity and description, not in its base features (kleiveist, 2026k, 2026s). The browser build and desktop application thus use the same frontend and backend base.

*Full* does not include a persistence provider; `postgres` is the selectable server-side SQL extension (kleiveist, 2026l).

## 4.7 Optional Capabilities

The catalog contains six features. `tauri` requires `frontend`; `cloud` and `database` require `backend`; and `postgres` requires `database`. The directed rules in Figure 10 do not describe runtime communication (kleiveist, 2026g, 2026k).

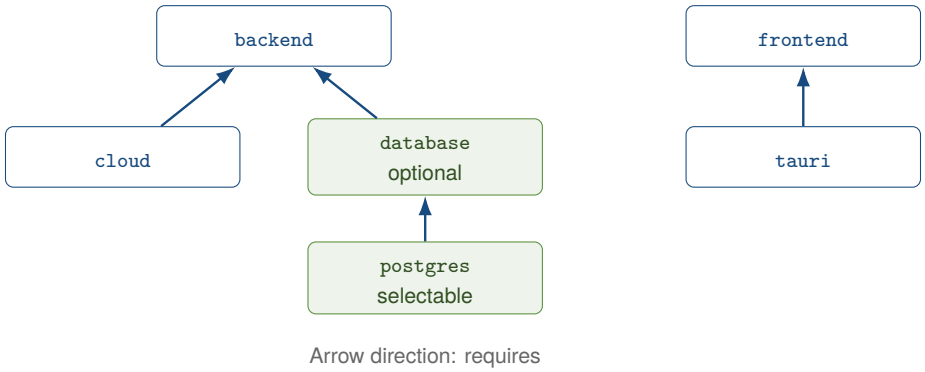


Figure 10: Implemented dependencies in the feature catalog

Only `postgres` is currently marked as a selectable capability. For backend profiles, it first resolves the optional, internal-only `database` feature and adds SQLAlchemy, Alembic, Psycopg, and tests. The PostgreSQL service is already declared as an inactive profile in the Compose document; the capability adds backend infrastructure and dependencies (kleiveist, 2026h, 2026s). Thus, `web-cloud + postgres` consists of the frontend, backend, cloud, database base, and PostgreSQL extension. The selection therefore does not merely activate the Compose entry; it adds the associated backend paths.

The resolver adds only optional prerequisites. It therefore rejects `web-only + postgres` and `desktop-local + postgres` because both lack a backend. Declaring new catalog entries alone does not make them implemented. They also require their own paths, dependencies, and tests.

Two implementation discrepancies remain: the dialog offers only `selectable` PostgreSQL, but a direct `--with database` argument accepts the database capability that is optional for backend profiles. In addition, contrary to the general profile documentation, entries for `.env.example` originate from `config/environment.toml`, not from `features.toml`. The presets remain unchanged, but the distinction between internal and selectable capabilities is not fully enforced.

### 4.8 Single-Repository Strategy

The five profiles are not template forks. The Core, feature implementations, profiles, and generator reside in a single maintenance base from which independent product projects are created (kleiveist, 2026k, 2026s).

The master also contains optional implementations and itself enables PostgreSQL. A generated project, by contrast, receives only the Core, resolved feature paths, manifest, frontend profile, and an appropriate `.env.example`. It then remains separate from the unchanged master. Centralized maintenance reduces parallel baselines; generated projects are not automatically synchronized back to the master. This separation protects the maintenance base from product changes. Conversely, existing generated products do not benefit from subsequent fixes to the master unless those fixes are incorporated deliberately.

## 5 Tooling and Development Workflow

### 5.1 Role of control.py

`tools/control.py` consolidates the public workflows. Its commands cover initialization, diagnostics, installation, development, configuration, database operations, quality, tests, builds, containers, Tauri, versioning, releases, and documentation. Help and group views structure this interface (kleiveist, 2026b, 2026m). The principal entry points include `init`, `doctor`, `install`, `run`, `quality`, `test`, and `build`. The `db`, `tauri`, `container`, and `release` groups add only the specialized workflows required in each case.

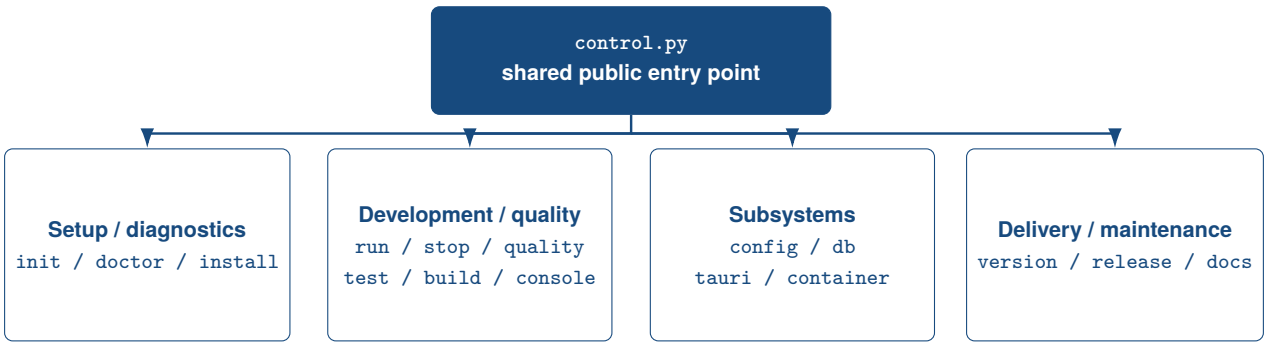


Figure 11: Command groups of the central project tooling

Source: Author's own illustration based on kleiveist (2026b), kleiveist (2026t), and kleiveist (2026m).

The governance commit split the dispatcher, which previously comprised 592 physical lines. In the examined state, `tools/control.py` comprises 197 lines and acts as a composition root; parser definitions reside in `tools/control_parser.py`, while the check itself is implemented in the modular `tools/quality/` package. That package has no reverse dependency on the dispatcher. The assessment therefore considers responsibility, coupling, and delegation rather than file length alone (kleiveist, 2026i).

With no action, `quality` is equivalent to the `check` subcommand. Focused actions are `size`, `complexity`, `architecture`, `lint`, and `format`; options include `--config` and `--format text|json`. Every action loads the policy, scans the repository, and validates exceptions. The complete action also combines size and complexity rules, architecture checks, language tools, compiler checks, and format checks.

Exit code 0 requires successful tool checks and no active error findings. An unsuppressed *ERROR* finding or a failed required tool produces 1. Parser, configuration, and scan errors produce 2, and keyboard interruption at dispatcher level produces 130. *INFO*, *WARNING*, and strong warning do not block by themselves. *INFO* is represented and counted in the JSON report, but no v1 rule produces it. External linter, compiler, and formatter errors appear as failed checks with tool output, not always as a structured finding with a CQ identifier.

Table 6 summarizes the 13 implemented rule identifiers. CQ severity depends on the applicable threshold; AR and EX findings are errors.

Table 6: Rule groups in Governance v1

| Group | Rule IDs | Subject | Severity |
|---|---|---|---|
| Code quality | `CQ001`, `CQ002`, `CQ101`–`CQ104`, `CQ201` | File, function, and class size, complexity, nesting, and parameters | threshold-dependent |
| Architecture | `AR001`–`AR004` | Layer and framework dependencies, feature internals, and router heuristic | ERROR |
| Exceptions | `EX001`, `EX002` | Invalid or expired exception entries | ERROR |

Source: Author's own compilation based on kleiveist (2026m).

Repository findings contain a path, optional line and symbol, ID, name, severity, actual value, threshold, detail, and remediation advice. Suppressed findings remain visible with their rationale; the summary counts severity levels and exceptions. The text report does not list INFO separately, whereas

the JSON report does. In addition, the `files_checked` count is in practice populated only by the size check, so focused complexity and architecture runs can report zero files despite examining source files. This reporting limitation does not alter finding status.

## 5.2 Project Initialization and Generation

`init` creates an independent product project from a profile and compatible capabilities. Selection can be guided or non-interactive; project identity, target, and dry run are configurable. A modified Tauri identity requires a valid reverse-domain identifier (kleiveist, 2026k, 2026t).

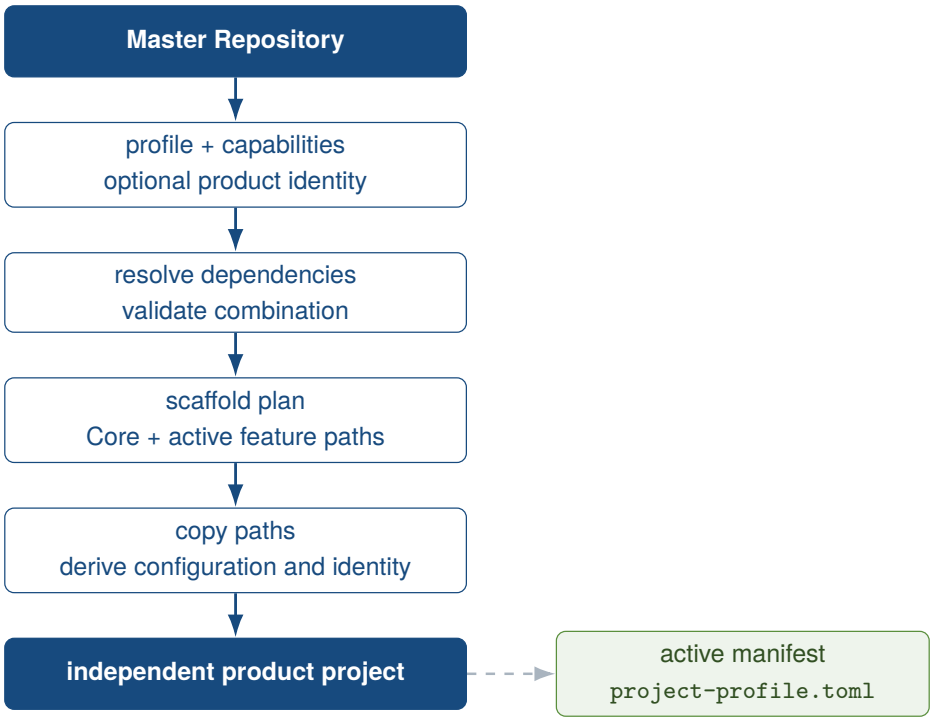


Figure 12: Workflow for profile-driven project generation

Source: Author's own illustration based on kleiveist (2026k) and kleiveist (2026t).

The generator resolves dependencies, creates the scaffold plan, and validates the sources and target before writing. It copies the Core and active features, generates the manifest, frontend profile, and `.env.example`, and adjusts existing product metadata. Transient directories are omitted; web profiles omit the Tauri script and CLI. When a product identity is specified, only frontend, backend, Compose, Tauri, and Cargo metadata that are actually present are adjusted. The scope thus remains dependent on the selected profile.

Within the master, targets are permitted only under `.generated/`; external targets are also possible. The repository itself and nonempty targets are rejected, and `--dry-run` writes nothing. One limitation remains: only the dialog filters by `selectable`; a direct `--with database` argument is accepted for backend profiles (kleiveist, 2026s). These preliminary checks keep the master and target separate and prevent partially generated projects in known conflict cases. The derivation is designed to be deterministic; nevertheless, the empirical evidence of reproducibility remains limited to the case examined.

## 5.3 System Check and Installation

The read-only `doctor` checks runtimes, configuration, paths, dependencies, and ports according to the profile. Disabled layers are not an error. In general, missing Docker produces only a warning; container, database, and Tauri prerequisites have their own stricter diagnostic paths (kleiveist, 2026q, 2026t).

`install` sets up only active components: frontend dependencies are preferentially installed from the lockfile, and backend dependencies are installed in a virtual environment. The dedicated `tools/.venv` is set up for repository tools, including Ruff, regardless of profile; the `backend/.venv` remains separate. Database packages and Chromium are added only when required; for Python, a pip/venv fallback is available alongside `uv`. A local `.env` remains unchanged. `tauri install` handles native operating-system and Rust prerequisites separately. Specifically, the frontend uses `npm ci` when a lockfile is present. An active backend receives its own environment and only the profile-dependent requirement sets. Chromium is installed only when Playwright is configured; disabled layers are skipped.

At the historical observation state `461fc751`, generated backend profiles had an installation discrepancy relevant to governance: `install` could provide Ruff through `backend/.venv` and omit `tools/.venv`, while the quality adapter expected Ruff there or on the global search path. In a freshly generated `web-cloud` profile, installation completed successfully, but the subsequent quality command reported Ruff as unavailable. Core CI avoided this combination through `install --skip-backend`; the generated backend-profile jobs did not. This commit-bound observation explains the profile failures at the time and remains part of the historical evidence (kleiveist, 2026f, 2026m).

In the v1.0.0 candidate state, the discrepancy is corrected: `install` provides `tools/.venv` regardless of profile, and the quality adapter uses this dedicated tooling runtime. The local `install` → `quality` path therefore has a uniform contract for Ruff resolution. This later correction does not rewrite the commit-bound CI results of the baseline.

The README requires Git, Python 3.11 or later, Node.js 24 or later, and Rust Stable for desktop profiles. The general Doctor does not check Git or these version ranges; the Tauri Doctor accepts any active Rust toolchain. CI, by contrast, explicitly sets up the specified major versions of Python and Node. Local enforcement therefore differs from the documentation (kleiveist, 2026e, 2026s). This limitation concerns how the prerequisite is checked, not whether the documented prerequisite applies.

## 5.4 Development Mode

Figure 13 shows the profile-dependent path from diagnostics and installation to quality assurance and delivery.

After a change, `quality` precedes tests and the build. Warnings remain visible in the report and feed back into the editing cycle without blocking by themselves; an active error terminates the command with exit code 1. This local entry point corresponds to the quality invocation in Core, profile, and release workflows, but does not assert identical execution environments.

`run` starts Vite and, only when the backend is active, Uvicorn as well. In the foreground, cancellation terminates the tracked process groups; in detached mode, PID and log data are stored. `stop` termi-

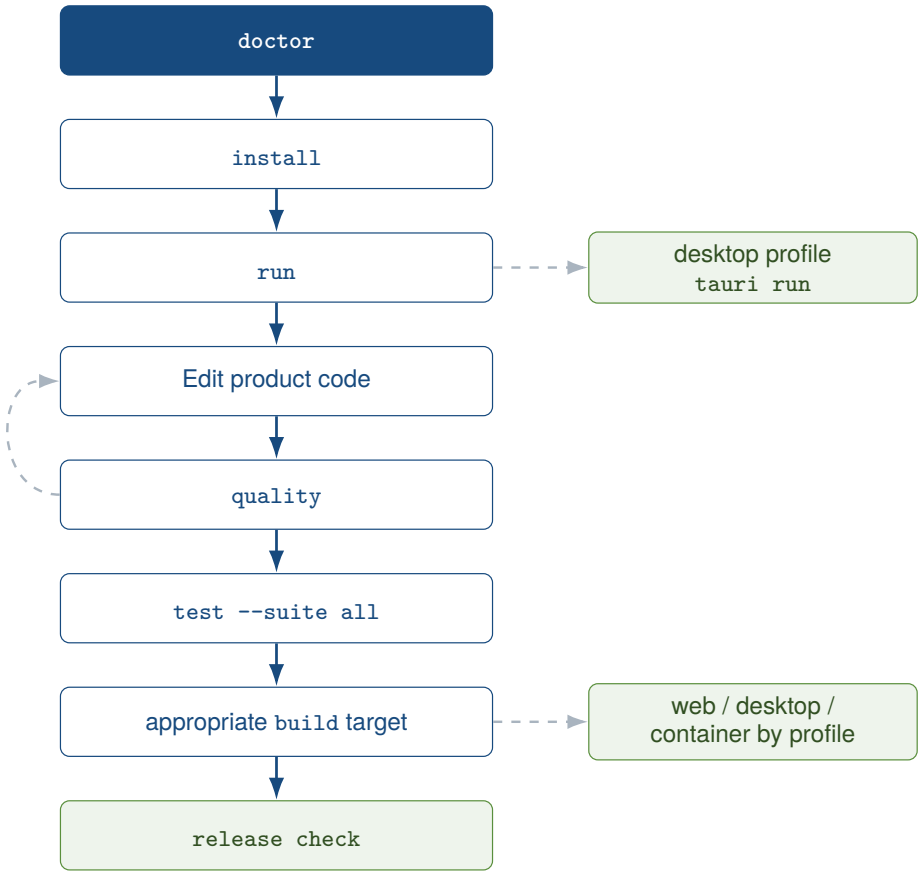


Figure 13: Basic structure of the development workflow

Source: Author's own illustration based on kleiveist (2026t), kleiveist (2026o), and kleiveist (2026c).

nates tracked processes and, by default, listeners on the configured ports as well; `--tracked-only` restricts this behavior to the tool's own tracked state (kleiveist, 2026b, 2026t). In the foreground, output from active services is combined. Detached mode, by contrast, retains process and log data under `tools/.runtime`; this state store provides the basis for `stop`.

The separate desktop path delegates to `tauri dev` and removes known server-side secrets from the child-process environment. It supports foreground and background operation. The console merely guides users through the same CLI workflows. The native development path thus remains separate from general Vite/Uvicorn operation without duplicating the command logic.

## 5.5 Tests and Builds

The governance separates repository-specific rules from established language tools. Scanning, severity, rule IDs, exclusions, exceptions, architecture rules, and the aggregate report remain in the shared quality package; linting, type checking, and formatting are delegated to the respective toolchains. Table 7 summarizes the coverage.

Table 7: Language-specific tools and boundaries in the quality gate

| Area | Tools | Automated | Principal limitation |
|---|---|---|---|
| Python | Tokenizer, Python AST, Ruff | LOC, function and class size, McCabe complexity, nesting, parameters, lint, and format | Syntax errors can omit scope metrics in a focused size run. |
| JS/TS | Dedicated LOC lexer, TypeScript AST, ESLint, TypeScript compiler, Prettier | LOC, classes and imports, function metrics, lint, types, and format | Architecture resolution covers only known relative and configured alias paths. |
| Rust/Tauri | LOC lexer, Syn AST analyzer (WASI), Wasmtime, rustfmt, Clippy, Cargo | File and function size, complexity, nesting, parameters, format, lint, compiler check | Syn analyzes syntax without type resolution or macro expansion; adapter semantics remain subject to review. |

Source: Author's own compilation based on kleiveist (2026m).

Ruff is declared as `>=0.14,<1.0`; version 0.16.4 was run locally. The frontend lockfile pins ESLint 9.39.5, Prettier 3.9.6, TypeScript 5.9.3, and typescript-eslint 8.67.0; the release workflows use Node 24. The Tauri application manifest names Rust 1.77 and edition 2021 as its minimum. Separately, the analyzer build environment pins rustc 1.97.1 and edition 2024. Its lockfile pins Syn 2.0.119 and proc-macro2 1.0.107, while the tooling environment pins Wasmtime 47.0.1. These versions improve snapshot traceability but remain time- and environment-dependent.

The scanner recognizes `.py`, `.ts`, `.tsx`, `.js`, `.jsx`, `.mjs`, `.cjs`, and `.rs`. Physical lines are distinguished from code LOC: blank lines and comment-only lines do not count, while code followed by a comment counts once. Python docstrings and other multiline strings count as code. For JavaScript/C-like syntax, a dedicated lexer handles block comments; the Rust LOC pass also handles nested block comments and raw strings. This lexer stage classifies code lines only. Rust scope and control-flow metrics are parsed separately from a complete syntax tree by the analyzer built with Syn 2.0.119. `.mjs` and `.cjs` receive file LOC and the usual applicable frontend checks, but no repository-specific scope metrics or architecture checks.

The checked-in analyzer targets `wasm32-wasip1` and is executed by Wasmtime 47.0.1 from the tooling

environment. The 535,787-byte artifact has SHA-256 `7a27cb02a9392b62c487b4ce73a03524d7260b804c0c49eb4520ceb1a1cacfd8`. Its provenance binds the exact rustc 1.97.1 compiler commit, target and dependency versions, and a source digest over `build.py`, `Cargo.toml`, `Cargo.lock`, `rust-toolchain.toml`, and `src/**/*.rs`.

The build removes inherited compiler, flag, target, and release-profile overrides. A versioned remapping contract canonicalizes the source root, user home, Cargo home and target, Rust sysroot, and every dependency root; broad mappings precede specific ones because the last matching rustc entry wins. Residual private physical build paths in the artifact cause failure. Before each run, the host checks the provenance and artifact. The WASI instance has bounded memory and fuel; malformed or unsupported syntax, invalid provenance, a corrupt artifact, resource exhaustion, and inconsistent JSON metrics abort the analysis. Generated profiles without a Rust toolchain can therefore run the same Rust analysis. The analyzer replaces neither rustc nor Clippy: it does not resolve types, expand macros, or check runtime semantics.

Excluded directories are `.cache`, `.dist`, `.generated`, `.git`, `.pytest_cache`, `.venv`, `__pycache__`, `coverage`, `dist`, `generated`, `node_modules`, `target`, and `vendor`; `Cargo.lock`, `package-lock.json`, and the configured paths `build/**`, `backend/build/**`, `frontend/dist/**`, `src-tauri/gen/**`, and `src-tauri/target/**` are also excluded. The general directory name `build` is intentionally not excluded globally, so handwritten tooling paths with that name remain visible. Exclusions define the scope, for example for generated or external artifacts; patterns that are too broad could hide relevant code.

Exceptions are separate, temporary deviations for handwritten files. Required fields are a known CQ or AR identifier, an exact relative path, a rationale of at least ten characters, and an ISO expiry date; an optional symbol is matched exactly. The file must exist, and duplicates and unknown IDs are rejected. The expiry date is inclusive; only the following day produces `EX002`, and a suppression remains visible in the report. The implementation does not, however, check whether the scanner covers the file or whether an optional symbol actually exists in the source. Further discrepancies are discussed in Section 7.3.

In the backend, a Python AST analysis maps imports to four layers: API, Application/Services, Domain, and Infrastructure. It detects configured forbidden directions, direct Domain dependencies on seven framework roots, and direct database imports in routers. Handlers above 50 code LOC also produce `AR004`. In the frontend, boundaries between Application, Features, API, UI, and Shared are checked, together with cross-feature imports of internals; the target feature's public root `index.*` remains allowed. Statically resolvable `import()` and `require()` calls are included; dynamic and transitive dependencies that cannot be resolved statically and unknown aliases remain outside the analysis. A router heuristic cannot fully identify semantic business logic; thin Tauri commands are a review-only rule.

The eight test suites cover tooling, schema, API, database, PostgreSQL, frontend, E2E, and Tauri/Rust. `test --suite all` provides the complete project-wide entry point; reports are optional (kleiveist, 2026b, 2026t). Among other elements, the tooling suite tests the generator, profiles, and configuration; the component suites delegate to Pytest, Vitest, Playwright, or Cargo. The test areas therefore remain independently executable.

Selection remains profile-dependent: if a feature is absent, the suite reports `SKIP`; if sources or pre-

requisites are missing for an active feature, it reports `FAIL`. PostgreSQL without a test URL and E2E without Playwright configuration are skipped. The runner can repair missing backend environments and manage the development services for E2E. This behavior deliberately distinguishes absent features from defective active components. A skipped test is therefore not evidence of success.

The build paths remain separate: web produces Vite output and a ZIP; desktop delegates the target and bundle to Tauri; and container builds frontend, backend, or both images for cloud profiles. `container validate` checks the Compose model without starting it (kleiveist, 2026d, 2026o). The web path additionally creates a ZIP package. Desktop builds remain tied to an active Tauri feature and the respective target platform.

GitHub Actions uses the same public entry points for the Core, profile matrices, PostgreSQL, and desktop targets. CI supplements these with operating systems, runtimes, and a temporary PostgreSQL service (kleiveist, 2026e). Local and external workflows therefore use comparable entry points, but do not run under identical environmental conditions.

## 5.6 Release and Packaging Workflow

`VERSION` is the central version source; validation and write-based synchronization are separate. `release check` does not produce artifacts. The command checks the version, product identity, Tauri rules, tag, and Git working tree. Inconsistencies, placeholders from the template, and a mismatched or modified state are errors. Missing signing configuration, by contrast, produces only a warning. Only the master repository accepts its canonical template identity (kleiveist, 2026b, 2026o).

Web delivers a Vite build and ZIP. On the respective host, Tauri produces native Windows, macOS, or Linux bundles; additional paths are available for a portable Windows ZIP and a Linux cross-build. CI produces only ephemeral, unsigned validation artifacts. These packages validate the technical packaging path. Without a signature and product-specific approval, they are not publishable releases.

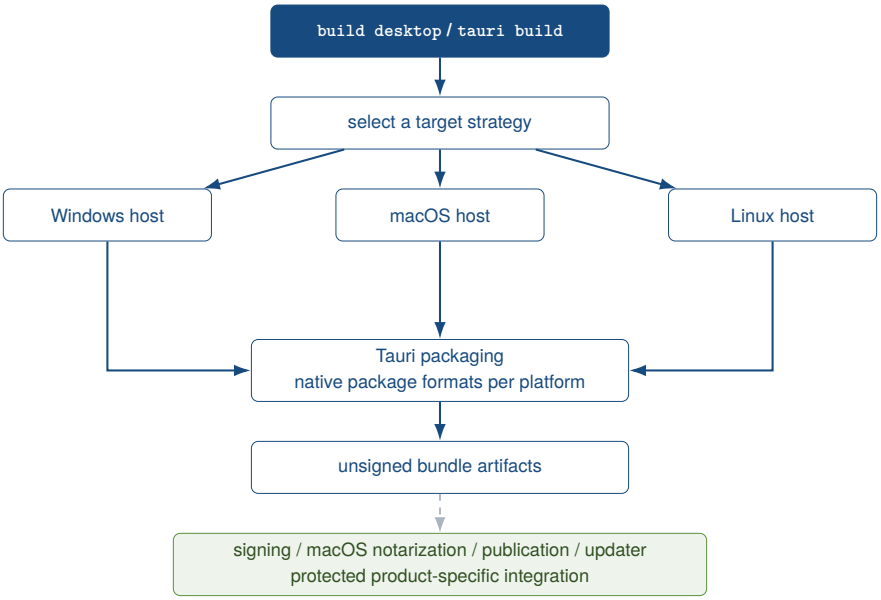


Figure 14: Cross-platform model for desktop releases

Source: Author's own illustration based on kleiveist (2026o).

Signing and notarization are product-specific. The same applies to publication, app stores, and updaters. The release workflow combines tests with web and container builds. These are followed by the gate and the unsigned desktop matrix. The workflow neither publishes nor deploys (kleiveist, 2026e, 2026o).

For cloud profiles, the container commands check Docker, Compose, and deployment files and build versioned images. Compose simulates production locally and optionally runs PostgreSQL. The foundation does not prescribe a provider and replaces neither secrets management nor a controlled migration job (kleiveist, 2026d, 2026h). A Docker/Compose foundation is therefore not equivalent to a complete architecture for AWS, Azure, or GCP.

### 5.7 Developer Onboarding

Onboarding follows the sequence of cloning the master, running `doctor`, running `init`, moving to the target project, running `doctor` again, and then running `install` and `run`. Product code, configuration, tests, and commits then belong in the derived repository. Template maintenance, by contrast, takes place in the master and validates the entire baseline there (kleiveist, 2026k, 2026s). This separation prevents product development from inadvertently modifying the shared template.

The basic prerequisites are Git, Python 3.11 or later, and Node.js 24 or later. Desktop profiles additionally require Rust Stable and platform-specific Tauri packages; Docker, PostgreSQL, and PyGitIndex are required only for the respective optional tasks. Optional tools therefore do not block every profile.

`AGENTS.md` also provides the repository policy to supporting coding agents. It defines 900 code LOC as a permitted absolute maximum, requires evaluation from 600 LOC and normally a split plan above 750 LOC, prohibits merely weakening rules, and requires thin routers, adapters, Tauri commands, and composition roots. After structural code changes, the complete quality gate is to run before handoff, followed by relevant tests. Changes that cross subsystem boundaries require the complete applicable suite. Remaining findings must be disclosed (kleiveist, 2026p).

The agent file is explicit normative context, not empirical evidence of effectiveness. Its existence proves neither that a particular model follows the rules reliably nor that agent-assisted code is automatically clean. Technically enforced rules, human review, and model behavior remain separate layers.

The README and specialized documents organize getting started, tooling, profiles, configuration, CI, containers, and releases into separate areas. This standardizes the public setup paths, but does not yet demonstrate a measurable reduction in onboarding time (kleiveist, 2026e, 2026o, 2026q, 2026t). The documentation structure thus supplements technical onboarding, but does not replace an empirical measurement of its duration.

## 6 Quality Assurance and Verification

### 6.1 Test Strategy

Verification cross-checks central observations against repository artifacts, executed checks, and commit-bound results. Test code alone does not demonstrate execution, just as a workflow file does

not demonstrate a successful CI run. The case study records local commands with the date, exit code, and result; the complete console output is not a versioned repository artifact. Execution results and the acceptance report therefore carry more weight than code, configuration, and documentation (ISO/IEC, 2023; Runeson & Höst, 2009).

Four levels cover the baseline: tooling tests check the CLI, profiles, generator, configuration, release, and repository; component tests cover the schema, backend, frontend, and Tauri/Rust. Integration tests exercise PostgreSQL 16 together with the configuration and Alembic. Build checks produce web, container, or native artifacts (kleiveist, 2026e).

Governance v1 adds an evidence chain comprising the policy, rule classification, synthetic boundary and architecture fixtures, exit-code tests, and the CI invocation. On 22 August 2026, the combined direct Pytest run over `tools/tests` and `case-study/tests` ended with 333 passed tests. Of these, 135 were focused quality and workflow tests. This result is separate from the public aggregate suite described later. Table 8 summarizes the tested transitions.

Table 8: Executed boundary tests for Governance v1

| Subject | Tested values | Expected transitions |
|---|---|---|
| File, code LOC | 599, 600, 601, 750, 751, 899, 900, 901 | PASS through 600; WARNING through 750; STRONG_WARNING through 900; then ERROR |
| Physical file lines | 1200, 1201 | PASS; WARNING |
| Function, code LOC | 50/51, 80/81, 120/121 | all four levels |
| Class, code LOC | 300/301, 500/501, 700/701 | all four levels |
| Complexity | 10/11, 15/16, 20/21 | all four levels |
| Nesting | 3, 4, 5, 6 | all four levels |
| Parameters | 5/6, 8/9, 10/11 | all four levels |
| Router, code LOC | 50, 51 | PASS; `AR004 ERROR` |

Source: Author's own execution against technical baseline `461fc751` with the present, not-yet-versioned retrospective documentation; test sources: kleiveist (2026n).

The LOC fixtures cover blank lines, comment-only lines, and trailing comments, together with block comments, Python docstrings, JavaScript/TypeScript comments, nested Rust comments, and raw strings. Empty files and files without a final newline were not tested separately. Exception tests cover valid, unknown, incomplete, expired, and duplicate entries and missing files; exceptions for excluded code and absolute paths are absent. Positive and negative backend and frontend architecture fixtures test constructed import relationships. These fixtures establish detection of their cases, not every possible semantic violation.

Ruff and ESLint adapter tests partially use simulated tool output; for Rust, the generated Clippy configuration is checked, but no dedicated negative boundary fixture is run. These gaps limit the integration evidence despite the passing tests.

The release verification on 23 August 2026 separately adds evidence for the final analyzer state. A total of 146 focused Python regression tests for Rust syntax, metrics, payload validation, the WASI host, and packaging passed; the analyzer crate passed its 13 Rust integration tests as well as formatting, Clippy, and the compiler check. Three relocated builds on the pinned Rust 1.97.1 Linux builder — the normal registry, alternate homes and targets with spaces, and an in-tree vendor — produced the same 535,787-byte `wasm32-wasip1` artifact with SHA-256 `7a27cb02a9392b62c487b4ce73a03524`

`d7260b804c0c49eb4520ceb1a1cacfd8`. This local release evidence closes the specifically identified analyzer gap, but rewrites neither the test counts nor the remote results of the historical baseline.

The scope follows the active profile. *PASS* denotes a successful check, and *FAIL* an executed check that failed. *Not verified* means that evidence is absent, while *out of scope* denotes a deliberate boundary; disabled items are not applicable. Generation and structure checks address A-01 and A-03, the public tooling interface addresses A-04, and valid and rejected capability combinations address A-07. Quality and architecture tests address A-08 through A-10. Figure 15 classifies the governance components and their local and CI-based control flow.

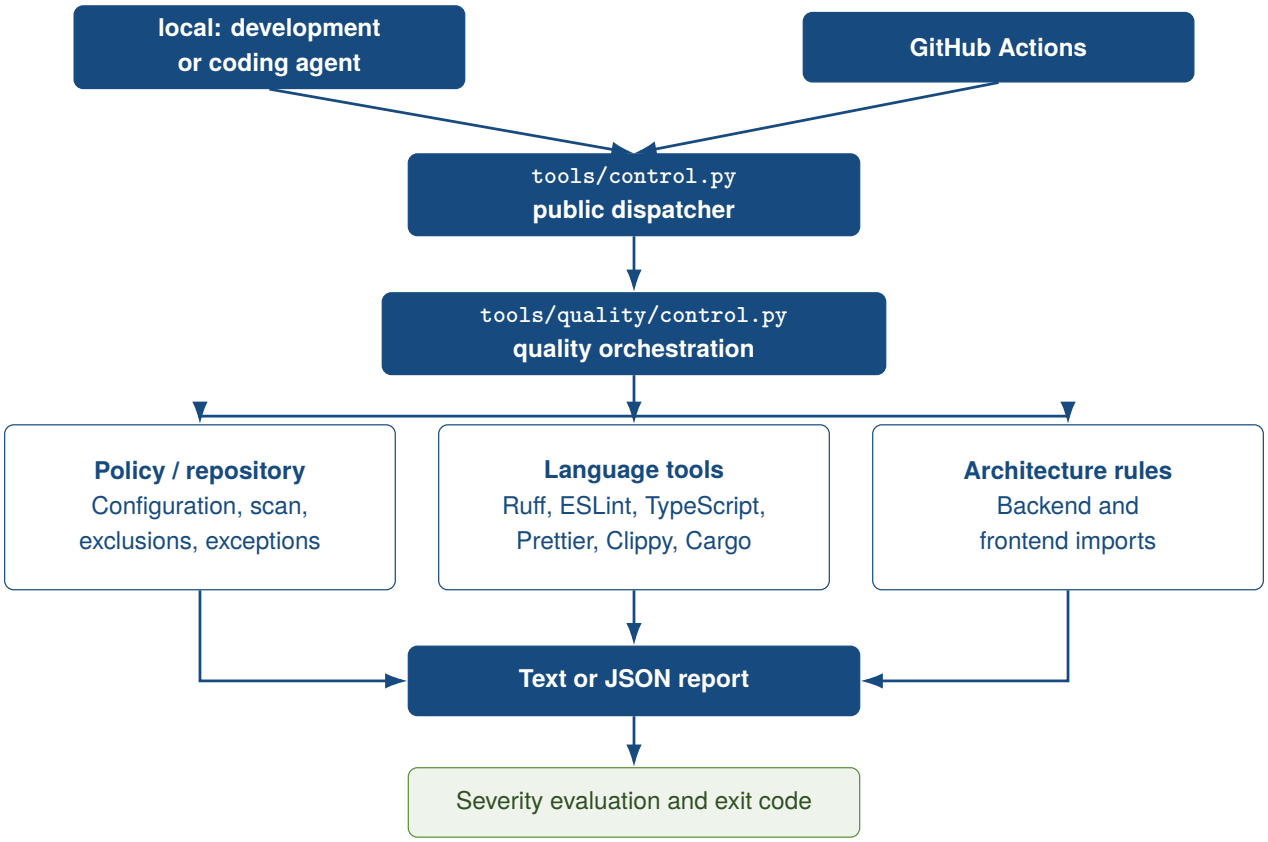


Figure 15: Quality governance as a development and CI layer outside the product runtime

Source: Author's own illustration based on kleiveist (2026m) and kleiveist (2026f).

## 6.2 Continuous Integration

Core CI, Profile Matrix, PostgreSQL Integration, and Desktop CI separately check the baseline, five profiles, PostgreSQL combinations, and native packages. Release Validation runs only manually or for version tags. Since Governance v1, Core CI has a preceding Ubuntu quality job for pull requests, pushes to `main`, and manual runs, with Python 3.11, Node 24, Rust 1.97.1, rustfmt, Clippy, caches, and Tauri system packages. It invokes `quality` through `tools/control.py`. `continue-on-error` is not set; tooling, backend, frontend, and container declare `needs: quality` (kleiveist, 2026f).

After generation and installation, the profile matrix runs the gate for all five scaffolds before tests and builds; the generated PostgreSQL path does so for three backend profiles. The release workflow runs quality before tests and artifacts. Desktop CI and the direct PostgreSQL job do not have their own gate. The sequence "quality before tests before build" therefore does not apply uniformly to every

workflow.

The current v1.0.0 workflow state pins Node 24 and Rust 1.97.1 and uses PostgreSQL 16.15 in image `16.15-alpine3.24` for integration jobs. Core CI and Release Validation additionally build or verify the `wasm32-wasip1` analyzer produced with Syn 2.0.119 and execute it with the pinned Wasmtime 47.0.1 runtime. Core and Release build and compare the artifact on Ubuntu; Desktop CI only executes the checked-in artifact on Linux, macOS, and Windows and does not rebuild it there. This workflow configuration describes the current gate; a remote PASS must still be demonstrated for the final commit.

The runs dated 7 August and queried on 9 August 2026 correspond to HEAD `a4487ac` and are not entirely green. Core CI succeeded for the backend, schema, frontend, web build, Compose, and both images; tooling, profiles, and configuration failed. In the profile matrix, both web profiles passed, while the three desktop profiles failed during the project test. PostgreSQL integration succeeded for the master and web-cloud, but not for the two desktop combinations. Desktop CI packaged on Linux and macOS; Windows failed during frontend installation (GitHub, 2026a, 2026d, 2026f, 2026h).

The runs evaluated on 22 August 2026 for `461fc751` show another mixed result. In the Core run, the new quality job, including the unchanged-working- tree check, succeeded; backend, frontend, and container passed, while tooling, profile, and configuration tests caused the overall workflow to fail. In the profile matrix, `web-only` passed; `desktop-local` initially passed quality and later failed the project test. `web-cloud`, `desktop-cloud`, and `full-platform` failed during quality. The direct PostgreSQL integration job passed, while the three generated PostgreSQL profiles failed during quality. Desktop CI built on Linux and macOS, while Windows again failed during frontend installation (GitHub, 2026b, 2026e, 2026g, 2026i).

Public metadata establishes job and step status; log archives were unavailable without authorization. The fresh local reproduction described in Section 5.3 for `461fc751` plausibly explains the profile failures at the time through Ruff resolution. This is an inference from implementation, workflow, and reproduction, not a direct quotation from CI logs. The green Core quality job therefore establishes the blocking gate in the master workflow at the time, not profile compatibility of the later corrected candidate state.

Locally, on 9 August 2026, all 194 tooling tests as well as the schema, API, and SQLAlchemy tests passed on `a4487ac` under Python 3.13.14. These local results neither supersede the failed external jobs nor provide evidence for the paths that were not executed because Node.js, Rust, and Docker were unavailable. Release Validation was not run for this commit.

## 6.3 Acceptance and Technical Verification

The final acceptance report rates commit `1cdfb6e` as *PASS WITH OPEN EXTERNAL ITEMS* and *READY FOR CASE STUDY*. This applies to the case study baseline, not to production readiness or external deployments. On this commit, 189 tooling tests passed (kleiveist, 2026r).

The acceptance process generated and installed all five profiles, executed profile-dependent diagnostics, tests, and web builds, and packaged the three desktop profiles as Linux DEBs. With Compose available, strict validation passed. The three backend profiles additionally passed with PostgreSQL, including the connection, migrations, tests, and build; the desktop variants were packaged again. As expected, the two incompatible PostgreSQL combinations were rejected before the target was gener-

ated (kleiveist, 2026r).

The governance checks were executed locally again against technical baseline `461fc751` and the present retrospective documentation. `quality size`, `quality complexity`, and `quality architecture` ended with exit code 0. The complete report selected 123 source files and contained 42 warnings, eight strong warnings, no active rule-based error, and no suppression. The aggregate command nonetheless ended with exit code 1 because Clippy and `cargo check` could not find the local system libraries `javascriptcoregtk-4.1` and `webkit2gtk-4.1`. This is not a complete PASS. For the executed repository rules, it establishes only the absence of an active ERROR finding; warnings do not block by definition.

The public `doctor` also ended with 1. A required `DATABASE_URL` and the backend environment were missing, while Docker was additionally reported as a warning. `test --suite all` repaired the backend environment and passed tooling, schema, API, database, and frontend; PostgreSQL and E2E were skipped, and Tauri failed because `javascriptcoregtk-4.1` was absent. The aggregate status remained 1. This result must be read separately from the direct Pytest evidence of 333 passing tests.

`build web` ended with 0 and produced Vite output and a ZIP. `docs index --dry-run` also ended with 0 but reported ten index, 26 documentation-link, one README, and one AGENTS backlink changes. The dry run therefore establishes a functioning preview, not documentation that was already synchronized. All local results are dated 22 August 2026; technical source files were not changed during their execution.

Figure 16 and Table 9 distinguish the earlier local acceptance, historical CI on `a4487ac`, and the governance addendum for `461fc751`. Later failures do not retroactively invalidate earlier commit-bound evidence, but they prevent its use as a fully green finding for a different state.

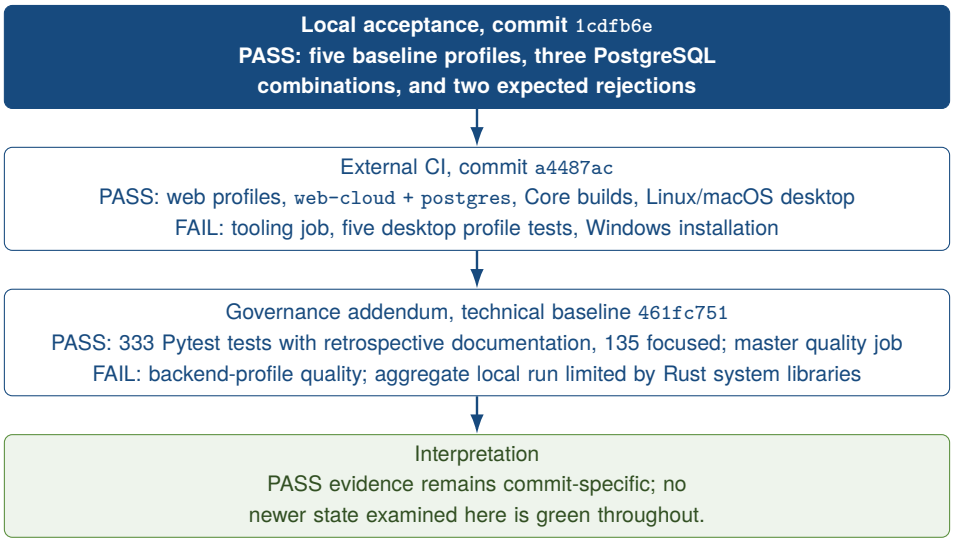


Figure 16: Profile verification by evidence state and commit

Source: Author's own illustration based on kleiveist (2026r), kleiveist (2026s), and the evaluated GitHub Actions runs.

Table 9: Profile-specific technical verification

| Profile / combination | Gen. | Install / Doctor | Tests | Web | Desktop | PG / migr. | CI on `a4487ac` |
|---|---|---|---|---|---|---|---|
| `web-only` | P | P | P | P | – | – | P |
| `web-cloud` | P | H | P | P | – | – | P |
| `desktop-local` | P | P | P | P | P (Linux DEB) | – | F: profile test |
| `desktop-cloud` | P | H | P | P | P (Linux DEB) | – | F: profile test |
| `full-platform` | P | H | P | P | P (Linux DEB) | – | F: profile test |
| `web-cloud + postgres` | P | P | P | P | – | P | P |
| `desktop-cloud + postgres` | P | P | P | P | P (Linux DEB) | P | F: profile test |
| `full-platform + postgres` | P | P | P | P | P (Linux DEB) | P | F: profile test |

**Legend:** P = PASS; H = PASS WITH NOTE; F = executed check failed; – = not applicable. The six central result columns are taken from the local acceptance on `1cdfb6e`; the final column reports the profile-specific GitHub Actions run on `a4487ac`. `web-only + postgres` and `desktop-local + postgres` were correctly rejected before target generation in the local acceptance (P). Source: kleiveist (2026r) and GitHub (2026f, 2026h).

**Governance addendum `461fc751`:** The master quality job, `web-only`, and initially `desktop-local` passed the gate; `desktop-local` then failed during the project test. `web-cloud`, `desktop-cloud`, `full-platform`, and the three generated PostgreSQL profiles failed during quality. These results supplement the historical final column; they do not replace it because they belong to a different commit. Sources: the governance CI runs in Section 6.2.

## 6.4 Reproducibility

Two generations of `desktop-cloud + postgres` on `1cdfb6e` were byte-for-byte identical for identical inputs. `init --dry-run` wrote nothing; nonempty targets and incompatible capabilities were rejected before writing. This evidence applies only to this combination and environment (kleiveist, 2026r; Lamb & Zacchiroli, 2022).

Frontend, Cargo, and three Python lockfiles pin dependencies; npm and Cargo use locked versions, and the container baseline additionally uses hashes. The hash-verified installation process was tested (kleiveist, 2026d, 2026e). The operating system, toolchain, registries, network, and platform SDKs nevertheless limit reproduction; native results remain platform-specific.

For the governance, the versioned TOML policy, fixed rule IDs, boundary tests, and the same public quality entry point locally and in CI improve traceability. This does not mean that every environment produces the same aggregate result. In the historical evidence run, the complete local run failed on native Tauri libraries while the Core quality job passed; generated backend profiles on `461fc751` failed because Ruff resolution differed at the time. Policy, classification, and the tested boundaries are thus the most reproducible elements. Complete tool execution remains dependent on installation, tool versions, and system libraries; the v1.0.0 candidate state now has the uniform tooling-runtime contract from Section 5.3.

The final Rust analyzer binds its build and runtime more tightly: provenance binds the exact rustc 1.97.1 compiler commit, target, and dependency versions. Its `source_sha256` hashes only `build.py`, `Cargo.toml`, `Cargo.lock`, `rust-toolchain.toml`, and `src/**/*.rs`; Wasmtime 47.0.1 is pinned exactly for execution. A versioned contract canonicalizes source, home, Cargo-target, Cargo-home, Rust-sysroot, and dependency paths after the build removes inherited overrides. Broad mappings precede more specific mappings.

Three relocated builds — normal registry, alternate homes and targets with spaces, and an in-tree vendor — were byte-for-byte identical on the pinned Rust 1.97.1 Linux builder. Core and Release check this reproduction on Ubuntu; macOS and Windows only execute the checked-in artifact. The evidence explicitly does not claim byte identity across operating systems or with future compilers.

### 6.5 Security and Configuration Boundaries

Tooling tests provide evidence for configuration precedence and for validation of environments, ports, and CORS origins. They also check the profile-dependent `.env.example` and ensure that `DATABASE_URL` is present only for PostgreSQL. These tests passed locally, while the external tooling job failed on the same commit (GitHub, 2026a; kleiveist, 2026q).

Only `VITE_API_BASE_URL` may reach the frontend. Tests reject secrets designated as public, remove `DATABASE_URL` from the frontend and Tauri, and mask database passwords in diagnostics. No database variable is created without PostgreSQL. Additional checks cover health, readiness, error responses, database-independent liveness, the Tauri CSP, and capability metadata (kleiveist, 2026o, 2026r).

Thus, only the configuration, secret, and desktop boundaries of the baseline have been verified. Authentication, authorization, production credentials, TLS, and provider-specific hardening remain product-specific; a complete security assessment has not been demonstrated.

### 6.6 Outstanding External Checks

Table 10 distinguishes failed checks, missing evidence, and deliberate product boundaries. Failed CI jobs are real errors; a Compose stack that was not started, by contrast, is not verified.

The Core run on `461fc751` provides evidence for a green master quality gate, Compose validation, and both images, but not for `docker compose up` or production deployment. Generated profiles with a backend do not have green quality evidence. Locally, Rust/Tauri checks remain incomplete because system libraries are missing; PostgreSQL and E2E were skipped without a service or configuration. These cases are FAIL or SKIP, not PASS.

Linux and macOS packages were built unsigned on `461fc751`; Windows was not. Branch Protection could not be read because the required API permission was unavailable. The successful documentation dry run still reported index and backlink changes to synchronize. Signing, notarization, updaters, production credentials, and authentication/RBAC require a product and external infrastructure (GitHub, 2026b, 2026e; kleiveist, 2026o).

Table 10: Outstanding and external verification items on `461fc751`

| Verification item | Status | Reason | Required evidence |
|---|---|---|---|
| Backend-profile quality | FAIL | three profiles and three PostgreSQL variants fail during quality; Ruff resolution reproduced locally | corrected installation and a green matrix run on a fixed commit |
| Local Rust/Tauri path | FAIL | `javascriptcoregtk-4.1` and `webkit2gtk-4.1` are missing | complete quality and Tauri run with native packages |
| PostgreSQL and E2E locally | SKIP | test URL and Playwright configuration are absent | executed integration and E2E tests |
| Documentation synchronization | open | dry run reports 38 index, link, and backlink changes | synchronized documentation and another dry run |
| Windows package | FAIL | frontend installation failed in the native Windows job | successful native build and artifact upload |
| Release Validation | not verified | no manual or tag run is available | successful non-publishing run on a fixed commit |
| Compose startup | not verified | only the model and master images were checked in CI | `docker compose up`, health/readiness evidence, and controlled teardown |
| Branch Protection | not verified | the public API requires authentication | authorized evidence of the required checks for `main` |
| Signing, notarization, updater | outside the scope | product-specific identity and protected credentials are intentionally absent | protected native release pipeline for each target platform |
| Production operation and auth/RBAC | outside the scope | cloud environment, business model, and security model are product-specific | product-specific acceptance including credentials and deployment |

Source: Author's own compilation based on kleiveist (2026o, 2026r), kleiveist (2026m), and the evaluated GitHub Actions runs for the governance version.

# 7 Evaluation of the Technology Template

## 7.1 Productivity Effects

The evaluation compares the requirements, implemented mechanisms, and evidence. The criteria are reuse, standardization, variability, maintainability, testability, reproducibility, developer experience, extensibility, and platform coverage. Complexity, operational maturity, and documentability supplement this assessment. The project's internal acceptance report provides project-specific evidence, but no independent replication (kleiveist, 2026r). This distinction keeps observed mechanisms, executed verification activities, and inferred effects separate. Automation alone is not considered a measurable productivity gain.

Here, productivity encompasses the effort required to obtain a runnable initial project, recurring setup decisions, the development flow, and subsequent maintenance. A single activity metric does not represent it adequately (Forsgren et al., 2021). Conceptually, the effect can therefore be expressed as

$$\textit{Productivity benefit} = \textit{avoided project effort} - \textit{template maintenance effort} - \textit{additional complexity}$$

In the absence of time and effort measurements, it cannot be quantified. The relationship also shows that centralized maintenance and additional complexity can reduce the project effort avoided.

At project initialization, `control.py` combines profile selection, generation, system checks, and instal-

lation. The acceptance results provide evidence for these steps across all five profiles and for the rejection of incompatible PostgreSQL selections (kleiveist, 2026r). The mixed CI results for the later HEAD limit the transferability of this evidence. Less manual setup is therefore plausible, but a specific amount of time saved has not been demonstrated.

During development, profile-dependent entry points standardize startup, testing, and builds. The shared frontend, configuration contract, and clear responsibilities encourage reuse; specialized tools and platform prerequisites remain necessary. Governance v1 adds an earlier, standardized feedback channel for defined size, complexity, and dependency findings. Central rule IDs and remediation advice can make findings easier for people and coding agents to classify. Neither shorter remediation times nor lower review effort were measured, however. The 42 warnings and eight strong warnings in the local report are a snapshot of policy application, not a productivity metric.

Central changes benefit future projects, but require shared maintenance of profiles, generator, tests, and documentation. Projects that have already been generated are not updated automatically. The benefit is thus distributed across project initialization, ongoing development, and subsequent baseline maintenance. It is particularly plausible when several technically related projects use the same foundations.

Quantitative evidence is lacking. A future controlled evaluation would need to compare similar projects with and without the template in terms of initialization and onboarding time, setup errors, failed builds, proportion of reuse, and maintenance time. It should also record the time to the first domain-specific feature and the number of manual setup decisions. Only such comparative data would permit a statement about the magnitude of the effect.

## 7.2 Strengths

The shared baseline combines centralized maintenance with clear boundaries between the frontend, backend, desktop, persistence, and tooling layers. The shared frontend addresses A-02 and A-05 without burdening every profile with all components. This separation supports independent tests and product-specific extensions without duplicating the shared presentation layer.

Five profiles represent recurring platform configurations. Dependencies prevent the use of PostgreSQL without a backend. The acceptance results for all base profiles and for the permitted and prohibited PostgreSQL combinations support this model for the limited current catalog, but not for an arbitrary number of future capabilities (kleiveist, 2026r).

`control.py` standardizes the entry point while keeping the specialized tools visible. The manifest, configuration contract, tests, and build paths increase traceability. The byte-identical generation of two `desktop-cloud + postgres` projects provides concrete evidence of this, albeit limited to one combination and environment (kleiveist, 2026r). Profile-dependent skipping of absent layers keeps the shared command interface aligned with the actual project scope.

An additional strength is that Governance v1 operationalizes selected rules that had previously been documented. Central policy, 13 stable rule identifiers, severity and exception models, text/JSON reports, and 135 focused tests are implemented. The successful Core quality job shows that the master path can block errors through the public exit code. Separating dispatcher, quality package, and standard tools keeps verification outside the product runtime (GitHub, 2026b; kleiveist, 2026m). This

strength concerns reproducible rule detection, not a guarantee of general code quality.

Table 11 contrasts the strengths with the limits of their applicability.

Table 11: Evidence-based comparison of strengths and limitations

| Area | Strength and evidence | Limitation and evidence | Significance |
|---|---|---|---|
| Architecture | Separate frontend, backend, Tauri, and persistence boundaries; a shared frontend. | Shared schemas are not consumed automatically by the frontend and backend. | Reuse with clear responsibilities; contract changes remain manual. |
| Variability | Five declarative profiles; dependencies and three permitted PostgreSQL combinations have passed acceptance. | Only one selectable capability; the direct CLI does not strictly enforce the distinction between `optional` and `selectable`. | Controlled selection in the current catalog, with limited implications for extensions. |
| Tooling | Shared profile-dependent entry point for diagnostics, installation, startup, testing, and building. | Several specialist tools and native prerequisites remain necessary. | Fewer distinct interaction paths, but no technology-independent workflow. |
| Quality | Central policy, 13 rule IDs, exception and architecture tests; 135 focused tests and a green master quality job. | Generated profiles with a backend fail during quality; the aggregate local run is limited by Rust system libraries; `CQ001` is suppressible. | Defined violations are detected more reproducibly; general code quality or maintainability is not guaranteed. |
| Reproducibility | Two identically configured projects were byte-for-byte identical in the documented experiment. | Evidence covers only one combination and one examination environment. | Specific evidence without claiming complete cross-platform reproducibility. |
| Operation and platforms | Web and image builds, PostgreSQL integration, and unsigned Linux/macOS packages are documented. | Compose startup, production cloud operation, a Windows package, signing, and notarization remain outstanding. | Template maturity must not be equated with production readiness. |
| Maintenance | Core, profiles, tooling, tests, and documentation reside in one baseline. | Changes may affect multiple profiles; product projects receive no automatic updates. | Central maintenance is consolidated, but requires broad regression testing and transfer processes. |

Source: Author's own evaluation based on Chapters 3–6, the acceptance report, and the current CI runs (GitHub, 2026a, 2026b, 2026d, 2026f, 2026h, 2026i; kleiveist, 2026m, 2026r).

## 7.3 Weaknesses and Limitations

Technical weaknesses, deliberate scope boundaries, and unverified areas must be distinguished. Only the first category directly denotes a disadvantage of the implementation. Although scope boundaries create a need for adaptation, they do not constitute an unmet requirement. A lack of verification does not yet permit either a positive or a negative technical judgment.

The normal file-size classification is unambiguous: 900 code LOC are permitted and produce a *STRONG_WARNING*; 901 code LOC produce `CQ001 ERROR`. The project documentation defines 900 LOC as an absolute maximum. The list of permitted exceptions technically also includes `CQ001`, however; an exactly matching entry can suppress the 901-LOC error and make the gate pass. There is no integrated regression test for the complete path from 901 LOC through `CQ001` and an exception to gate status. The policy is therefore intended and documented as absolute, but is suppressible in the examined state. This discrepancy must neither be hidden by a blanket success claim nor by modifying the technical project during the retrospective analysis.

There are further validation boundaries for exceptions. Leading slashes are removed before the absolute-path check, so absolute POSIX and Windows paths are normalized instead of rejected as documented. An exception for an existing but excluded or unscanned file remains valid; optional symbols are not checked against source text. Expiry dates, unknown IDs, duplicates, and missing files are tested and handled as errors. Time limits and rationales support governance but cannot automatically prevent repeated extensions or excessively broad exclusions; review remains necessary.

LOC, scope size, and cyclomatic complexity are structural indicators, not measurements of comprehensibility, cohesion, or domain correctness. Import analysis captures direct, statically resolvable relationships, not every dynamic, transitive, or semantic architecture violation. The frontend resolver ignores unknown aliases; router logic below the size threshold and thin Tauri adapters remain partly or wholly matters for review. A file below 900 LOC can therefore still be structurally problematic.

At the historical observation state, profile applicability followed the files present rather than the manifest, and the reporter had no dedicated SKIP status. Other limitations were differing Ruff resolution in generated backend profiles and outdated public documentation paths: README and the tooling guide did not fully name the quality entry point, and the CI documentation still described the older job structure. The dry run confirmed pending index and backlink changes. These findings limited the clarity and profile maturity of the otherwise central governance. In the v1.0.0 candidate state, the Ruff discrepancy in particular is corrected by the consistently provided `tools/.venv`; the historical finding and its CI results at the time remain unchanged.

Consolidation in the master repository increases internal complexity: changes to the core, generator, or catalog require regression tests across several profiles. In addition, the frontend and backend do not use the shared JSON schemas at runtime; contract changes remain manual. Direct CLI selection distinguishes `optional` and `selectable` less clearly than the dialog does. Both discrepancies create risks of maintenance errors or incorrect operation. The evidence also conflicts by commit: on `a4487ac`, web profiles and local tooling tests passed while external tooling, desktop-profile, and Windows checks failed. On `461fc751`, the master quality gate passed, while backend-profile gates and Windows did not. These findings do not retroactively call earlier commit-specific acceptance into question, but they prevent a fully green assessment of the governance state.

The deliberate exclusions are a backend in `web-only`, a cloud API in `desktop-local`, and PostgreSQL without a backend. Domain logic, authentication, role models, mandatory persistence, cloud providers, and native domain-specific commands likewise remain product-specific. The new decision framework for product persistence provides no universal local-file, embedded-database, or synchronization implementation. This deliberate scope boundary is an extension path, not an implementation defect (kleiveist, 2026l).

Compose startup, production deployment, Release Validation, and Branch Protection have not been conclusively verified. Unsigned Linux and macOS packages were built, while Windows packaging failed. Signing, notarization, publication, and production cloud operation remain product- or infrastructure-specific (GitHub, 2026b, 2026e; kleiveist, 2026r). The packaging results therefore demonstrate neither signed releases nor production cloud operation. The subject of the evaluation is the template baseline, not the production readiness of a product. The remaining risks therefore primarily concern operations, platforms, and release processes.

## 7.4 Maintenance Effort

The single-repository strategy consolidates the core, features, profiles, generator, tests, and documentation. This avoids parallel baseline changes and benefits future projects. Repositories that have already been generated must, however, incorporate fixes independently. The source of truth thus reduces the number of starting points that require maintenance, but not the effort needed to migrate existing products.

Central maintenance encompasses several languages, dependency managers, toolchains, platforms, and release cycles. This diversity follows from the web, backend, desktop, and deployment objectives, but increases the expertise required, the breadth of testing, and dependency on the environment. Native builds and external services are more maintenance-intensive than a purely web-based, technologically homogeneous starter.

Five profiles, six features, and one selectable capability currently limit the combination space; acceptance covers five base paths and three permitted PostgreSQL extensions (kleiveist, 2026r). Additional capabilities expand both the dependencies and the test matrix. A combinatorial explosion has not been demonstrated, but increasing maintenance effort is plausible. With such extensions, the catalog, resolver, configuration contract, test matrix, and documentation must remain mutually consistent.

Central maintenance is therefore more demanding than maintenance of a small, homogeneous starter, but can consolidate repeated maintenance of the baseline. Its economic viability depends on frequency of use, similarity between projects, and maintenance time; cost data are unavailable.

## 7.5 Comparison with Alternative Template Strategies

The architectural strategies are compared in terms of reuse, consistency, variability, entry complexity, maintenance, testability, and adoption of updates (Clements & Northrop, 2001; Krueger, 1992). The qualitative levels do not form an overall ranking; high entry complexity, for example, is a disadvantage.

*Separate template repositories* remain individually manageable, but can diverge. A *copy-and-paste starter* minimizes template logic, but provides no channel through which subsequent improvements can be propagated. GitHub templates accordingly create a separate, unrelated history (GitHub, 2026c). Separate repositories therefore give greater weight to organizational isolation than to the central consistency of shared components.

*Framework-specific starters* such as `create-vite` offer a simple entry point into a narrow area. The backend, desktop layer, persistence, and overall workflow must be added separately when needed (VoidZero Inc. and Vite Contributors, 2026). For a small, homogeneous frontend, this lower entry complexity may be more appropriate than the full-stack approach under examination.

A *monolithic universal template* simplifies selection, but burdens small projects with unused components. The examined *single repository with profiles and capabilities* combines centralized maintenance with selected paths. This increases the complexity of the master repository, resolver, and test matrix; updates do not reach generated projects automatically. It thus combines centralized variability before generation with the autonomy of a copied starter afterward.

The profile-based approach is particularly suitable for recurring, technically related projects. Small individual projects may benefit from framework starters, while organizationally separate product lines may benefit from separate repositories. There is no universal winner, but rather a set of different trade-offs.

## 7.6 Overall Evaluation

Table 13 distinguishes criteria that are largely or partially addressed from those that have not been conclusively demonstrated. These levels are qualitative and measure neither quality nor productivity

Table 12: Qualitative comparison of template strategies

| Strategy | ongoing reuse | Consistency | Variability | Entry complexity | Maintenance consolidation | Updates to existing projects |
|---|---|---|---|---|---|---|
| Separate template repositories | medium | medium | medium | low | low | manual for each template or project |
| Copy-and-paste starter | low after copying | low | high | low | low | no automatic adoption |
| Framework-specific starter | high within a narrow scope | high within a narrow scope | low to medium | low | high within the starter | usually manual after generation |
| Monolithic universal template | medium | high | low | high | high | depends on the selected derivation method |
| Single repository with profiles | high within the defined scope | high | high, controlled | medium | high | centralized in the master, manual in derived projects |

Source: Author's own qualitative comparison based on Krueger (1992), Clements and Northrop (2001), GitHub (2026c), and VoidZero Inc. and Vite Contributors (2026).

numerically.

Table 13: Consolidated evaluation of the technology template

| Criterion | Evidence base | Assessment | Key limitation |
|---|---|---|---|
| Reuse and standardization | Shared Core, five profiles, generator, and uniform command interface | largely addressed | Applies to the defined stack; derived projects are not updated automatically. |
| Variability and extensibility | Profile presets, capability dependencies, and verified PostgreSQL combinations | largely addressed | Small current catalog; extensions increase the maintenance and testing scope. |
| Testability and reproducibility | Documented profile, integration, and build acceptance, plus a byte-for-byte identical generation comparison | partially addressed | Some current remote jobs failed; reproducibility experiment covers only one combination. |
| Developer experience and documentability | `control.py`, diagnostic paths, versioned manifests, and structured documentation | partially addressed | Onboarding and interaction effort were not measured empirically; multiple toolchains remain visible. |
| Maintainability | One central baseline with shared tests and documentation rules | partially addressed | Technology diversity, variant regression, and manual interface synchronization. |
| Code-quality and architecture governance | Central policy, 13 rule IDs, 135 focused tests, local commands, and the master quality job | partially addressed | Generated backend profiles are not green; the complete local run is limited by Rust system dependencies; `CQ001` is suppressible and a passing gate is not evidence of clean code. |
| Platform coverage and operational maturity | Evidence covering web, backend, PostgreSQL, container images, Linux, and macOS; provider-neutral boundary | not conclusively demonstrated | Windows package, Compose startup, signing, notarization, and production deployment have not been evidenced. |

Source: Author's own evaluation based on the preceding chapters, kleiveist (2026r), and the CI runs evaluated in Section 6.2.

The governance extension improves the technical enforcement of selected code- quality and architecture rules. Central configuration, stable rule identifiers, boundary, exception, and architecture tests, and local and CI entry points are implemented. The combined run with 333 passing tests, 135 of them focused quality and workflow tests, and the successful master quality job support this assessment.

The improvement remains limited: the complete local run failed because of Rust system libraries, quality steps in generated backend profiles are not green, and the normative 900-LOC limit is technically suppressible. The rules also measure selected structural indicators, not long-term maintainability or clean code. The governance is therefore substantially expanded, but not yet conclusively verified across profiles, tools, and platforms.

The earlier acceptance supports the technical viability of the defined profiles for the paths examined there. The core, presets, dependencies, and tooling support reuse, standardization, and flexibility. Generation, installation, tests, builds, PostgreSQL combinations, and Linux packaging are substantiated (kleiveist, 2026r). External runs on `461fc751` add a green master quality gate and evidence for web, container, Linux, and macOS paths; backend-profile quality, further project tests, and Windows remain unsuccessful (GitHub, 2026b, 2026e, 2026i). The evidence varies in scope by commit and platform.

Limitations include the complexity of central maintenance, manually synchronized contracts, and the absence of updates for generated projects. Windows packaging, signed releases, notarization, and production cloud operation have not been demonstrated. Deliberately omitted provider and domain logic, by contrast, is not a defect. Table 13 makes the distinction between an addressed requirement and the outstanding scope of verification explicit.

Automation and shared workflows provide plausible, but unquantified, potential for productivity gains. The current state is suitable as a standardized foundation for the intended project families, but not as a finished production application or a universal template. Whether the central investment is economically worthwhile remains unresolved in the absence of usage and cost data.

# 8 Application Scenarios and Transfer to Customer Projects

## 8.1 Suitable Project Types

Suitability depends on access channels, the backend and persistence, deployment, native integration, and infrastructure, security, and operational needs. The evidence covers combinations that can be generated and verified profile paths. By contrast, the levels *well suited*, *conditionally suited*, and *not primarily intended* qualitatively assess the distance from the baseline, not the probability that a project will succeed (kleiveist, 2026k, 2026r). The more central requirements lie outside the available components and evidence, the greater the adaptation effort and remaining risk.

`web-only` is suitable for browser-based frontends without a template backend. `web-cloud` adds a central API and provider-neutral deployment structure, for example for internal business and data applications. PostgreSQL remains optional; authentication, authorization, domain models, and production infrastructure are product-specific. The profile therefore covers the initial technical structure, not the particular business application.

`desktop-local` is appropriate for local tools without a central backend, provided that native functions can be added within a manageable scope; a local database is absent. `desktop-cloud` adds central services, and `full-platform` also adds the browser channel. The primary application domain therefore comprises web-, cloud-, and desktop-oriented business and full-stack applications. For all desktop variants, more extensive native functions must be added and verified specifically.

| Category | Project types | Decisive criterion |
|---|---|---|
| WELL SUITED | Browser frontend; web + backend/cloud; local desktop tool; desktop + cloud; web + desktop + cloud | Access channels and runtime layers match a verified profile; domain-specific and operational functions still need to be added. |
| CONDITIONALLY SUITED | high proportion of native code; highly customized infrastructure; stringent compliance or security requirements | A profile covers only part of the requirements; substantial extensions or additional evidence are required. |
| NOT PRIMARILY INTENDED | hard real-time systems; game-engine-based games; native mobile apps; core firmware and embedded systems | The underlying runtime or platform architecture falls outside the Vite/FastAPI/Tauri baseline. |

Figure 17: Qualitative suitability overview by architectural fit and adaptation effort

Source: Author's own transfer assessment based on the profile definitions and technical acceptance (kleiveist, 2026k, 2026r).

Figure 17 and Table 14 summarize the classification. *Suitable* means only that the baseline covers essential platform components. Domain logic, user interface, integrations, product security, and operations remain open, as does the economic assessment.

Table 14: Mapping of technical project types to the template baseline

| Project type | Suitability | Profile | Required additions and limitations |
|---|---|---|---|
| Browser frontend | well suited | `web-only` | Domain logic and UI; external APIs are possible, but the baseline provides no FastAPI backend. |
| Web application with API/cloud | well suited | `web-cloud` | Authentication, domain models, and production operation; PostgreSQL only if relational persistence is required. |
| Local desktop tool | well suited | `desktop-local` | Native commands, packaging, and signing are product-specific; no local database is included. |
| Desktop application with central backend | well suited | `desktop-cloud` | Production API, security, and signing; PostgreSQL remains optional. |
| Browser + desktop + cloud | well suited | `full-platform` | Channel-specific UX and tests; the profile sets no persistence provider, while PostgreSQL is optional. |
| Application with a high proportion of native code | conditionally suited | desktop profile or another foundation | Substantial effort for platform APIs, permissions, system-level services, or a native UI. |
| Highly specialized infrastructure | conditionally suited | depends on the partial architecture | Suitable only if the client or an individual service can be meaningfully isolated; infrastructure is not prescribed. |
| Hard real-time systems, game-engine-based games, native mobile applications, or core embedded systems | not primarily intended | no profile | The underlying runtime and platform requirements fall outside the baseline. |

Source: Author's own qualitative mapping based on kleiveist (2026k, 2026l, 2026r).

## 8.2 Unsuitable and Conditionally Suitable Project Types

Projects requiring extensive operating-system integration, a platform-specific UI, system-level services, or drivers are only conditionally suited. The Tauri shell does not provide sufficient evidence

of an adequate native foundation for such projects. Highly distributed infrastructure and specialized event-streaming or HPC requirements may likewise require a different architecture; at most, the baseline remains usable for a clearly delimited part.

Hard real-time systems, graphically intensive games, firmware, embedded systems, and native mobile applications do not belong to the primary target domain. Ordinary live updates do not turn a business application into a real-time system. Individual mobile or hardware-oriented functions are not excluded, but the template does not provide their technical foundation. The decisive question is therefore not whether a single characteristic is present, but whether the baseline supports the technical core of the product.

High security or compliance requirements do not preclude use of the baseline, but require additional evidence concerning protection requirements, audits, certification, and legal compliance. Compose validation and master images are evidenced, but Compose startup and production deployment are not (GitHub, 2026a). Linux and macOS packages were built without signing; the Windows packaging path failed (GitHub, 2026d). Signing and notarization remain product-specific (kleiveist, 2026o). The required TypeScript, Python, Rust, database, and deployment expertise must also be assessed. If this evidence or expertise is unavailable, the project is at most conditionally suited.

## 8.3 Analysis of Customer Requirements

The analysis begins with the functional objective, user groups, workflows, and deployment environment. Only then are the requirements classified in technical terms. Shared data processing, for example, may require a backend and persistence, but does not yet determine the use of PostgreSQL.

Browser and desktop access, server functions, persistence, offline needs, native functions, external APIs, operations, deployment, security, and compliance are recorded separately. Despite lacking a cloud backend, `desktop-local` includes neither a local persistence provider nor a local data model or synchronization function. Table 15 consolidates the guiding questions.

Table 15: Compact checklist for technology-neutral requirements analysis

| Area | Guiding question | Influence on selection |
|---|---|---|
| Domain functionality and use | Which workflows, user groups, and locations of use must be supported? | Determines the product scope; does not yet determine a profile. |
| Access channels | Will the product be used in a browser, as a desktop application, or through both channels? | Distinguishes web, desktop, and full-platform paths. |
| Central services | Must functions or shared data be available on the server side? | Requires a backend-capable cloud profile. |
| Persistence and offline operation | Which user data is created; which is structured and which consists of files or assets; what is the source of truth; are offline operation, relational queries, or synchronization required? | Determines the provider, schema, migration, backup, and recovery independently of the profile; PostgreSQL requires a backend. |
| Native integration | Which operating-system APIs, devices, or platform-specific interfaces are required? | Determines the need for Tauri and the distance from the native baseline. |
| Integration and operation | Which external systems, target environments, and scaling and operational requirements exist? | Produces product-specific adapters and infrastructure decisions. |
| Security and compliance | Which protection, evidence, audit, or regulatory requirements apply? | Requires additional measures and evidence independently of the profile. |

Source: Author's own compilation.

The result comprises platform components and gaps, for example in authentication, data models, integrations, observability, backups, infrastructure, or signing. Only then can a decision be made between a suitable profile, a limited extension, and a solution outside the baseline. The method therefore deliberately follows the sequence of requirements, technical classification, and only then profile or technology selection.

## 8.4 Selecting the Project Profile

The access channels determine the profile: `web-only` denotes a browser without a backend, `web-cloud` a browser with a backend, `desktop-local` a desktop application without a cloud backend, `desktop-cloud` a desktop application with a backend, and `full-platform` the browser and desktop channels together. There is no profile for a backend without provider-neutral deployment files.

The smallest suitable profile is selected. PostgreSQL is then assessed as a separate dimension and added to a backend profile only when needed. This is not permitted for `web-only` and `desktop-local` (kleiveist, 2026k).

The documentation records whether the profile is a complete match, requires specified extensions, or fails to meet central requirements. In the last case, a different architecture or clearly delimited partial use should be examined instead of selecting the largest profile. Team expertise and economic constraints supplement this technical decision, but do not change the meaning of the profiles.

## 8.5 Generating a Customer Project

`init` creates a separate project from the profile, capability, and project identity. Its active manifest is named `project-profile.toml`. Generation is the handover point to product development, not the completion of the product (kleiveist, 2026k, 2026t). For desktop projects, these inputs include a valid application ID.

In the target directory, `doctor`, `install`, `run`, and `test --suite all` check the baseline. Profile-dependent Tauri, database, or container checks supplement the transfer shown in Figure 18.

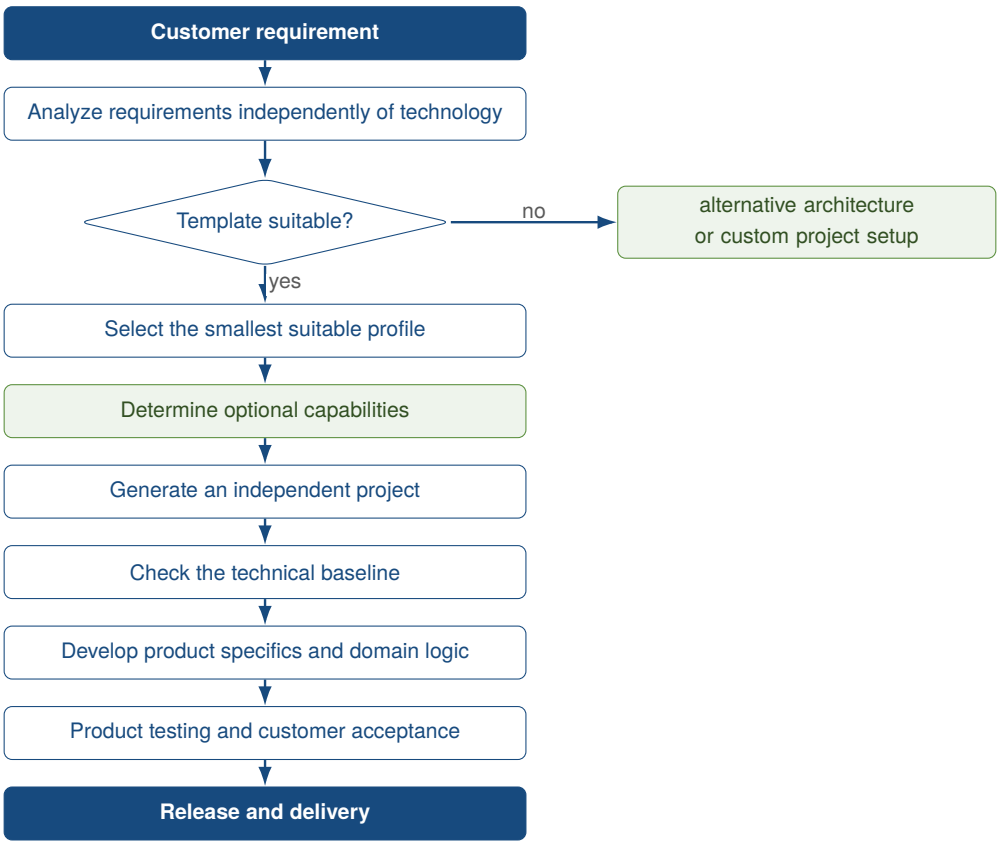


Figure 18: Vertical transfer process from customer requirement to product project

Source: Author's own illustration.

## 8.6 Transition to Product Development

After generation, the independent product project decides on domain models, user data, persistence providers, assets, schemas and migrations, backup and recovery, the security model, and, where applicable, synchronization (kleiveist, 2026l). Business rules, the user interface, integrations, operations, scaling, monitoring, data protection, and signing also remain product tasks. The baseline structures this work, but does not solve these product-specific tasks.

The master repository and the product have separate lifecycles. There is no permanent update or synchronization mechanism. Changes to the master repository must be transferred to products deliberately; product changes should flow back into the baseline only after a generalization and verification review.

Template acceptance and product acceptance remain separate. The commit-specific acceptance provides evidence for generation, installation, tests, and selected baseline builds, but not for deployment, signing, or functional and security requirements (kleiveist, 2026r). The product requires its own tests and evidence for these areas. They must cover the specific customer requirements, integrations, and operating conditions.

# 9 Conclusion

## 9.1 Summary of Results

The master template addresses repeated setup effort and diverging project bases through a shared core and declarative variability. It consolidates generation, configuration, boundaries of responsibility, workflows, and maintenance.

Five profiles combine the shared Vite frontend with a profile-dependent FastAPI backend and an optional Tauri layer; PostgreSQL can be selected only with a backend. `control.py` standardizes the development path. Generated projects remain independent and receive no automatic updates from the master repository.

Commit `461fc751` extended the previously mostly documented code-quality and architecture governance with central policy, 13 rule IDs, severity and exception models, scanners, language adapters, architecture checks, structured reports, focused quality commands, and CI enforcement. The normal classification of tested boundary cases, including 900/901 code LOC, and the detection of constructed architecture violations are substantiated. The gate remains outside the product runtime, and the dispatcher delegates checks to a dedicated package.

The local acceptance of an earlier commit supports the profile, PostgreSQL, test, web build, and Linux packaging paths. With the governance commit as the technical baseline, the direct Pytest run passed 333 tests, including 135 focused quality and workflow tests. The case-study tests checked the present, not-yet-versioned retrospective documentation at the same time. The web build and master quality gate also passed. Generated backend-profile gates, the local Rust/Tauri path, and Windows remained unsuccessful or incomplete; further evidence is unavailable or product-specific. The result establishes a baseline with controlled variability and stronger technical rule enforcement, not clean code, production readiness, long-term maintainability, or empirical suitability for customer projects.

## 9.2 Answers to the Research Questions

**Research Question 1: Requirements Addressed.** The template addresses the functional and technical, quality, and operational requirements through a baseline that can be generated, clearly defined components, profiles, shared workflows, centralized maintenance, and versioned configuration. Since Governance v1, selected size, complexity, and dependency rules are also operationalized through central policy, the local CLI, and CI. Separate runtime and verification paths support this; domain logic, authentication, authorization, and production operation remain product-specific; complete semantic architecture verification remains a review responsibility.

**Research Question 2: Variants for Web, Cloud, and Desktop.** Five presets combine the shared frontend with backend, cloud, and Tauri components; PostgreSQL is added only when backend capability is present. Generated projects predominantly contain the paths they require. `desktop-cloud` and `full-platform` are generated identically at the technical level and differentiated semantically.

**Research Question 3: Reuse and Standardization.** Centralized maintenance of implementations, profiles, the generator, configuration, tests, and documentation promotes reuse and consistent initial

states across projects. The benefit applies to the baseline and future generations; existing product projects receive no automatic updates.

**Research Question 4: Reduction of Recurring Effort.** Automated or standardized profile selection, generation, diagnostics, installation, startup, testing, and builds provide plausible potential for reducing recurring effort. Quality checks, rule IDs, and remediation advice standardize feedback for people and coding agents. Time, effort, and comparative data are lacking, however; neither a productivity improvement nor a lower defect rate has been demonstrated when central maintenance and multiple toolchains are taken into account.

**Research Question 5: Suitability for Project Types and Customer Requirements.** The template is suitable as a foundation for related web, desktop, and combined business applications whose platform and cloud needs can be represented. Persistence needs, persistence providers, and the source of truth are determined separately for the specific product. Highly native applications, games, and specialized real-time systems lie outside the primary target domain. The qualitative classification is not a universal assurance of suitability.

**Research Question 6: Limitations, Maintenance Effort, Risks, and External Dependencies.** Limitations include the growing breadth of regression testing, manual contract synchronization, insufficiently strict separation between optional and selectable capabilities, the absence of product updates, and the effort of maintaining multiple toolchains. Governance-specific limitations include technically suppressible `CQ001` findings, incompletely validated exceptions, heuristic architecture rules, missing negative Rust fixtures, and file-based rather than manifest-based profile derivation. Generated backend profiles, the local Rust/Tauri path, and Windows are not green; Release Validation, Compose startup, documentation synchronization, and Branch Protection are missing or open. Signing, notarization, the updater, production deployment, and the security model remain product tasks.

## 9.3 Critical Reflection

The project-specific case study has limited generalizability. The documentation, acceptance, and local verification originate from the project context; the developer perspective may influence the selection of evidence. Literature and official documentation are no substitute for independent replication. Commit-specific findings remain valid, but their transfer to other repository states or projects is limited.

**Construct validity.** Code LOC, function and class size, and complexity represent selected structural properties. They measure neither domain comprehension, cohesion, nor correct distribution of responsibilities. Import rules do not capture every dynamic, transitive, or semantic architecture violation. In particular, 900 code LOC is a generous project-specific maximum, not a universal maintainability criterion.

**Internal validity.** The evaluation originates within the project. Boundary and architecture fixtures may be tailored to the implemented rules. Simulated Ruff and ESLint outputs and the missing negative Rust fixture limit the integration evidence. Integrated tests for the 901-LOC exception through gate

status and for absolute and excluded exception paths are also absent. Passing tests do not rule out errors outside the tested cases.

**External validity.** The subject is one profile-based template. Thresholds and architecture mapping cannot readily be transferred to small starters, large enterprise systems, or generated codebases. The principles of central policy, declarative variability, and shared verification entry points are more transferable than the concrete numeric values.

**Reliability.** Results depend on the commit, tool versions, operating system, package registries, and native libraries. Divergent master and profile results and the locally failed Rust/Tauri path demonstrate this dependency. Without coverage measurement, Compose startup, and successful platform runs, there is no comprehensive evidence of quality or production readiness.

**Long-term effect.** The study establishes technical detection of defined violations at the time of observation. Maintenance effort, defect rate, refactoring frequency, development time, and model compliance were not measured longitudinally. The quality gate therefore guarantees neither clean code nor a causal improvement in long-term maintainability.

## 9.4 Outlook

In the v1.0.0 candidate state, the historical Ruff, exception, and documentation discrepancies are corrected: `tools/.venv` is profile-independent, exception boundaries are enforced, and documentation synchronization is checked. As the next release evidence, the master, all five profiles, PostgreSQL, Rust/Tauri, and Windows must be checked on the same final commit. Compose startup with health and readiness checks, Release Validation, and Branch Protection should supplement the technical evidence without claiming production readiness.

In the medium term, additional real negative Ruff, ESLint, and Rust fixtures beyond the regression cases for v1.0.0 would be useful. Existing JSON reports could be extended with SARIF, pull-request annotations, and trends. False positives and the appropriateness of thresholds should be observed by profile or project type. Other possible baseline extensions remain automated shared contracts, stricter capability selection, and controlled synchronization of generated projects.

Specific products still require signing, notarization, an updater, credentials, deployment, authentication, and RBAC. In the long term, a comparative study across projects should measure onboarding, time to the first build, setup errors, refactoring frequency, defects, and maintenance effort. A controlled comparison of agent-assisted development with and without the gate could test whether technical rule detection has effects beyond the observed snapshot.

## 9.5 Conclusion

The template is a centrally maintained, profile-based baseline for the defined web, cloud, and desktop variants. The shared core, capabilities, and tooling standardize recurring project workflows.

The governance extension closes a previous enforcement gap for selected code and architecture

rules. It makes policy centrally configurable and findings traceable; unsuppressed `ERROR` findings normally block locally and in CI. Controlled variability and an early feedback channel provide plausible potential for productivity gains. Without empirical data, the magnitude of that potential and suitability for customer projects can be substantiated analytically only for the delimited target domain.

The evidence applies only to the examined technical baseline. Important local checks passed, but governance and CI results remain mixed. Findings from parts of the policy can be suppressed through exceptions. External checks and product-specific release and security tasks remain open. The result therefore guarantees neither clean code nor a finished, cross-platform-verified product.

### Release-candidate validation addendum for v1.0.0

This addendum, dated 23 August 2026, preserves `461fc751` as the historical observation baseline; the commit-bound matrices and open items remain historical snapshots and do not anticipate later release results. For version 1.0.0, the disclosed governance gaps are candidate criteria: a `CQ001` `ERROR` above 900 LOC can no longer be suppressed, absolute exception paths and paths outside the scan are rejected, the dedicated tooling environment provides Ruff, and structural parity plus source/PDF provenance are checked automatically. All public quality, test, and build entry points must pass on the candidate state.

For scope and control-flow metrics in Rust, the candidate state uses an analyzer artifact for `wasm32-wasip1`. It was built with rustc 1.97.1 and Syn 2.0.119. Wasmtime 47.0.1 executes it with resource limits. The source state, Cargo lockfile, artifact, and ABI provenance are validated before analysis. This later evidence does not alter the historical findings.

The final commit hash is absent to avoid self-reference. No `v1.0.0` tag or GitHub Release is published at validation time. After successful exact-HEAD validation, GitHub Actions results and the final operator report must identify the exact candidate externally; a future tag and publication must bind to that same state.

## References


Bayer, M. (2026). *Alembic tutorial* [Alembic 1.19.1 Documentation]. Retrieved August 8, 2026, from https://alembic.sqlalchemy.org/en/latest/tutorial.html

Clements, P. C., & Northrop, L. M. (2001). *Software product lines: Practices and patterns*. Addison-Wesley Professional. Retrieved August 8, 2026, from https://www.sei.cmu.edu/library/software-product-lines-practices-and-patterns/

Forsgren, N., Storey, M.-A., Maddila, C., Zimmermann, T., Houck, B., & Butler, J. (2021). The space of developer productivity: There's more to it than you think. *Queue*, *19*(1), 20–48. https://doi.org/10.1145/3454122.3454124

GitHub. (2026a, August 7). *Core ci, run 31168215013* [GitHub Actions run for commit a4487ac in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/actions/runs/31168215013

GitHub. (2026b, August 17). *Core ci, run 32003918038* [GitHub Actions run for commit 461fc751]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/actions/runs/32003918038

GitHub. (2026c). *Creating a repository from a template* [GitHub Documentation]. Retrieved August 9, 2026, from https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-repository-from-a-template

GitHub. (2026d, August 7). *Desktop ci, run 31168214763* [GitHub Actions run for commit a4487ac in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/actions/runs/31168214763

GitHub. (2026e, August 17). *Desktop ci, run 32003917947* [GitHub Actions run for commit 461fc751]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/actions/runs/32003917947

GitHub. (2026f, August 7). *Postgresql integration, run 31168216208* [GitHub Actions run for commit a4487ac in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/actions/runs/31168216208

GitHub. (2026g, August 17). *Postgresql integration, run 32003918003* [GitHub Actions run for commit 461fc751]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/actions/runs/32003918003

GitHub. (2026h, August 7). *Profile matrix, run 31168215737* [GitHub Actions run for commit a4487ac in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/actions/runs/31168215737

GitHub. (2026i, August 17). *Profile matrix, run 32003918084* [GitHub Actions run for commit 461fc751]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/actions/runs/32003918084

ISO/IEC. (2023). *Systems and software engineering—systems and software quality requirements and evaluation (square)—product quality model* (International Standard ISO/IEC 25010:2023). International Organization for Standardization. Retrieved August 8, 2026, from https://www.iso.org/standard/78176.html

Klabnik, S., Nichols, C., & Krycho, C. (2026). *The rust programming language* [Official Rust Documentation]. Retrieved August 8, 2026, from https://doc.rust-lang.org/book/

kleiveist. (2026a, August 17). *Central code quality configuration* [Configuration at the examined commit]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/blob/461fc7519e0db638330904a7496c488e8a0d18bc/config/code-quality.toml

kleiveist. (2026b, August 17). *Central project control cli* [CLI implementation in examined commit 461fc751]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/blob/461fc7519e0db638330904a7496c488e8a0d18bc/tools/control.py

kleiveist. (2026c, August 17). *Code quality and architecture governance* [Project documentation at the examined commit]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/blob/461fc7519e0db638330904a7496c488e8a0d18bc/docs/def/code-quality.md

kleiveist. (2026d, August 7). *Container builds and local production simulation* [Project documentation in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/container-builds.md

kleiveist. (2026e, August 7). *Continuous integration* [Project documentation in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/ci.md

kleiveist. (2026f, August 17). *Core continuous integration workflow* [Workflow at the examined commit]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/blob/461fc7519e0db638330904a7496c488e8a0d18bc/.github/workflows/ci.yml

kleiveist. (2026g, August 13). *Database feature* [Project documentation in examined commit 461fc751; last substantive change 7257776]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/blob/461fc7519e0db638330904a7496c488e8a0d18bc/docs/def/database-feature.md

kleiveist. (2026h, August 6). *Deployment architecture* [Project documentation in the Template-Projekte repository]. Retrieved August 8, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/deployment-architecture.md

kleiveist. (2026i, August 17). *Enforce project-wide quality governance* [Governance commit with parent 0cbe1ba in the Template-Projekte repository]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/commit/461fc7519e0db638330904a7496c488e8a0d18bc

kleiveist. (2026j, August 13). *Framework architecture* [Project documentation in examined commit 461fc751; last substantive change 7257776]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/blob/461fc7519e0db638330904a7496c488e8a0d18bc/docs/def/architecture.md

kleiveist. (2026k, August 13). *Project profiles* [Project documentation in examined commit 461fc751; last substantive change 7257776]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/blob/461fc7519e0db638330904a7496c488e8a0d18bc/docs/def/project-profiles.md

kleiveist. (2026l, August 13). *Provider-neutral persistence architecture* [Project documentation in the Template-Projekte repository, commit 7257776]. Retrieved August 13, 2026, from

https://github.com/kleiveist/Template-Projekte/blob/7257776c21606f187156e1451ebe9fa3d851600c/docs/def/persistence-architecture.md

kleiveist. (2026m, August 17). *Quality engine implementation* [Quality package at the examined commit]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/tree/461fc7519e0db638330904a7496c488e8a0d18bc/tools/quality

kleiveist. (2026n, August 17). *Quality governance tests* [Governance-relevant test sources at the examined commit]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/tree/461fc7519e0db638330904a7496c488e8a0d18bc/tools/tests

kleiveist. (2026o, August 7). *Release and desktop packaging model* [Project documentation in the Template-Projekte repository]. Retrieved August 9, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/release-model.md

kleiveist. (2026p, August 17). *Repository guidelines for coding agents* [Agent guidelines at the examined commit]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/blob/461fc7519e0db638330904a7496c488e8a0d18bc/AGENTS.md

kleiveist. (2026q, August 13). *Runtime configuration* [Project documentation in examined commit 461fc751; last substantive change 7257776]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/blob/461fc7519e0db638330904a7496c488e8a0d18bc/docs/def/configuration.md

kleiveist. (2026r, August 7). *Template final acceptance* [Technical acceptance report in the Template-Projekte repository]. Retrieved August 8, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/dev/template-final-acceptance.md

kleiveist. (2026s, August 17). *Template-projekte: Examined state after code quality and architecture governance v1* [GitHub repository, version 0.1.0, commit 461fc7519e0db638330904a7496c488e8a0d18bc]. Retrieved August 22, 2026, from https://github.com/kleiveist/Template-Projekte/tree/461fc7519e0db638330904a7496c488e8a0d18bc

kleiveist. (2026t, August 6). *Tooling guide* [Project documentation in the Template-Projekte repository]. Retrieved August 8, 2026, from https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/tooling.md

Krueger, C. W. (1992). Software reuse. *ACM Computing Surveys*, *24*(2), 131–183. https://doi.org/10.1145/130844.130856

Lamb, C., & Zacchiroli, S. (2022). Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, *39*(2), 62–70. https://doi.org/10.1109/MS.2021.3073045

McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, *SE-2*(4), 308–320. https://doi.org/10.1109/TSE.1976.233837

Microsoft. (2026, July 27). *The typescript handbook* [TypeScript Documentation]. Retrieved August 8, 2026, from https://www.typescriptlang.org/docs/handbook/intro.html

PostgreSQL Global Development Group. (2026). *Postgresql 18.4 documentation*. Retrieved August 8, 2026, from https://www.postgresql.org/docs/current/index.html

Ramírez, S. (2026). *Fastapi features* [Official FastAPI Documentation]. Retrieved August 8, 2026, from https://fastapi.tiangolo.com/features/

Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, *14*(2), 131–164. https://doi.org/10.1007/s10664-008-9102-8

SQLAlchemy Authors and Contributors. (2026, June 15). *Engine configuration* [SQLAlchemy 2.0 Documentation, Release 2.0.51]. Retrieved August 8, 2026, from https://docs.sqlalchemy.org/en/20/core/engines.html

Tauri Contributors. (2026a). *Capabilities* [Tauri 2 Documentation]. Retrieved August 8, 2026, from https://v2.tauri.app/security/capabilities/

Tauri Contributors. (2026b, July 22). *What is tauri?* [Tauri 2 Documentation]. Retrieved August 8, 2026, from https://v2.tauri.app/start/

VoidZero Inc. and Vite Contributors. (2026). *Getting started* [Vite Documentation]. Retrieved August 8, 2026, from https://vite.dev/guide/